



# CIPHERBOARD

## Complete System Report

Hong Kong Diploma of Secondary Education  
Information and Communications Technology School-based Assessment

A production-oriented Mastermind platform

Web PWA | FastAPI | PostgreSQL | Redis | Supabase Auth | Python CLI

**Written from the perspective of the student developer**

System baseline: commit 3efefdb8937a798e699291cd39813fd03daa9d1e

Report date: 23 August 2026

Industry-standard technical report and implementation evaluation



# Student Declaration

I wrote this report to explain the design, implementation, verification, and operational readiness of Cipherboard. I wrote the report in the first person because it records my engineering decisions and what I learned while turning the Mastermind problem into a complete software system. I based technical claims on the repository at commit **3efefdb8937a798e699291cd39813fd03daa9d1e** and on verification commands that I ran on 23 August 2026.

I distinguish implemented behavior from design targets. For example, the product requirements define latency and Lighthouse targets, while the repository does not contain production percentile measurements. I therefore present those values as targets and do not claim that a live deployment has achieved them.

## FM.1 Document control

| Field          | Value                                               |
|----------------|-----------------------------------------------------|
| Document       | Cipherboard Complete System Report                  |
| Perspective    | Student developer, first person                     |
| Baseline       | Git commit 3efefdb8937a798e699291cd39813fd03daa9d1e |
| Report date    | 23 August 2026                                      |
| Classification | Technical project documentation                     |

# Abstract

---

Cipherboard is a server-authoritative Mastermind product that combines a responsive Next.js progressive web application, a versioned FastAPI service, a canonical Python rules engine, PostgreSQL persistence, Redis-backed coordination, Supabase identity, and a command-line interface. I designed the system so a player can begin with an anonymous authenticated account, play several solo and social modes, retain progress across refreshes, and enter ranked competition without allowing the browser to decide accepted feedback, score, or multiplayer outcomes.

My report follows the complete system from product requirements to production operations. I explain the duplicate-safe feedback algorithm, immutable attempt model, API transactions, real-time replay, encryption boundaries, data retention, accessibility, localization, deployment, and incident response. I also evaluate evidence from automated verification. In the checked environment, 133 Python tests passed and 8 PostgreSQL-dependent tests skipped; 47 JavaScript and TypeScript tests passed; strict TypeScript checking and Ruff linting passed; and the canonical engine achieved 100% measured branch coverage. The web unit suite reported 88.4% line coverage. The local Node runtime was version 20.20.0, below the pinned Node 24.18.0 requirement, so I treat the JavaScript results as useful development evidence rather than complete release evidence.

The strongest design choice is the separation between pure game rules and stateful delivery services. That boundary lets the API and CLI share one implementation while PostgreSQL remains authoritative for competitive outcomes. The main remaining work before public launch lies outside unit verification: production configuration, managed-service evidence, legal review, browser end-to-end execution in the pinned runtime, accessibility review, restore drills, load measurements, and a live two-client duel smoke test.

# Table of Contents

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>Student Declaration</b>                           | <b>1</b>  |
| FM.1 Document control . . . . .                      | 1         |
| <b>Abstract</b>                                      | <b>2</b>  |
| <b>Lists of Figures and Tables</b>                   | <b>7</b>  |
| <b>1. Executive Summary</b>                          | <b>8</b>  |
| 1.1 Key outcomes . . . . .                           | 8         |
| 1.2 Readiness judgment . . . . .                     | 8         |
| <b>2. Methodology and Evidence</b>                   | <b>9</b>  |
| 2.1 Evidence hierarchy . . . . .                     | 9         |
| 2.2 Commands executed for this report . . . . .      | 9         |
| 2.3 Assessment limits . . . . .                      | 9         |
| <b>3. Product and Stakeholder Context</b>            | <b>10</b> |
| 3.1 Product principles . . . . .                     | 10        |
| 3.2 Supported experiences . . . . .                  | 10        |
| <b>4. Requirements and Traceability</b>              | <b>11</b> |
| 4.1 Official preset rules . . . . .                  | 11        |
| <b>5. System Architecture</b>                        | <b>12</b> |
| 5.1 Repository scale . . . . .                       | 12        |
| 5.2 Version boundaries . . . . .                     | 13        |
| <b>6. Mastermind Domain Model</b>                    | <b>14</b> |
| 6.1 Feedback calculation . . . . .                   | 14        |
| 6.2 Scoring model . . . . .                          | 14        |
| <b>7. Canonical Game Engine</b>                      | <b>15</b> |
| 7.1 Duplicate-safe feedback implementation . . . . . | 15        |
| 7.2 Immutable guess transition . . . . .             | 16        |
| 7.3 Design evaluation . . . . .                      | 16        |
| <b>8. Game Modes and User Journeys</b>               | <b>17</b> |
| 8.1 Solo, practice, and custom play . . . . .        | 17        |
| 8.2 Daily challenge . . . . .                        | 17        |
| 8.3 Friend challenge . . . . .                       | 17        |
| 8.4 Private duel . . . . .                           | 17        |
| 8.5 Failure states . . . . .                         | 17        |
| <b>9. Web Application</b>                            | <b>18</b> |

|                                                       |           |
|-------------------------------------------------------|-----------|
| 9.1 Client boundaries . . . . .                       | 18        |
| 9.2 Game board implementation . . . . .               | 18        |
| 9.3 Responsive interaction . . . . .                  | 18        |
| <b>10. Progressive Web Application and Resilience</b> | <b>19</b> |
| 10.1 Cache and update boundary . . . . .              | 19        |
| 10.2 Degraded service . . . . .                       | 19        |
| <b>11. API and Backend Services</b>                   | <b>20</b> |
| 11.1 Request safety . . . . .                         | 20        |
| 11.2 Idempotent creation . . . . .                    | 21        |
| 11.3 Attempt transaction . . . . .                    | 22        |
| <b>12. Identity, Authentication, and Accounts</b>     | <b>23</b> |
| 12.1 Authorization . . . . .                          | 23        |
| 12.2 Account lifecycle . . . . .                      | 23        |
| <b>13. Data Architecture and Persistence</b>          | <b>24</b> |
| 13.1 Concurrency and invariants . . . . .             | 24        |
| 13.2 Redis boundary . . . . .                         | 24        |
| <b>14. Scoring, Rankings, and Achievements</b>        | <b>25</b> |
| 14.1 Deterministic order . . . . .                    | 25        |
| 14.2 Achievements . . . . .                           | 25        |
| 14.3 Anti-cheat position . . . . .                    | 25        |
| <b>15. Real-Time Multiplayer</b>                      | <b>26</b> |
| 15.1 Connection security . . . . .                    | 27        |
| 15.2 Winner and tie logic . . . . .                   | 27        |
| <b>16. Security Architecture</b>                      | <b>28</b> |
| 16.1 Secret protection . . . . .                      | 28        |
| 16.2 Residual assumptions . . . . .                   | 28        |
| <b>17. Privacy and Data Governance</b>                | <b>29</b> |
| 17.1 User controls . . . . .                          | 29        |
| 17.2 Retention and deletion . . . . .                 | 29        |
| 17.3 Processor governance . . . . .                   | 29        |
| <b>18. Accessibility, Inclusion, and Localization</b> | <b>30</b> |
| 18.1 Interaction contract . . . . .                   | 30        |
| 18.2 Localization architecture . . . . .              | 30        |
| 18.3 Release evidence needed . . . . .                | 30        |
| <b>19. Analytics, Administration, and Support</b>     | <b>31</b> |
| 19.1 Administrative controls . . . . .                | 31        |
| 19.2 Feature flags . . . . .                          | 31        |

|                                                         |           |
|---------------------------------------------------------|-----------|
| 19.3 Support boundary . . . . .                         | 31        |
| <b>20. Command-Line Application and SBA Component</b>   | <b>32</b> |
| 20.1 Educational value . . . . .                        | 32        |
| 20.2 Sample interaction . . . . .                       | 32        |
| <b>21. Configuration and Environment Management</b>     | <b>33</b> |
| 21.1 Production validation . . . . .                    | 33        |
| 21.2 Credential separation . . . . .                    | 33        |
| <b>22. Infrastructure and Deployment</b>                | <b>34</b> |
| 22.1 Release sequence . . . . .                         | 34        |
| 22.2 Rollback . . . . .                                 | 34        |
| <b>23. Observability and Operations</b>                 | <b>35</b> |
| 23.1 Health model . . . . .                             | 35        |
| 23.2 Alerts and dashboards . . . . .                    | 35        |
| <b>24. Reliability, Recovery, and Incident Response</b> | <b>36</b> |
| 24.1 Backup and restore . . . . .                       | 36        |
| 24.2 Incident response . . . . .                        | 36        |
| <b>25. Testing and Quality Assurance</b>                | <b>37</b> |
| 25.1 Unexecuted release checks . . . . .                | 38        |
| <b>26. Performance and Scalability</b>                  | <b>39</b> |
| 26.1 Scaling controls . . . . .                         | 39        |
| 26.2 Likely bottlenecks . . . . .                       | 39        |
| <b>27. Developer Experience and Maintainability</b>     | <b>40</b> |
| 27.1 Maintainability strengths . . . . .                | 40        |
| 27.2 Maintenance risks . . . . .                        | 40        |
| <b>28. CI/CD and Release Governance</b>                 | <b>41</b> |
| 28.1 Supply-chain controls . . . . .                    | 41        |
| 28.2 Go/no-go . . . . .                                 | 41        |
| <b>29. System Assessment</b>                            | <b>42</b> |
| 29.1 Key strengths . . . . .                            | 42        |
| 29.2 Main gaps . . . . .                                | 42        |
| <b>30. Recommendations and Roadmap</b>                  | <b>43</b> |
| 30.1 Suggested delivery order . . . . .                 | 43        |
| <b>31. Conclusion</b>                                   | <b>44</b> |
| <b>A. Appendix A - Repository Map</b>                   | <b>45</b> |

A.1 Primary implementation files . . . . . 45

**B. Appendix B - API Route Catalogue 46**

**C. Appendix C - Data Entity Catalogue 48**

**D. Appendix D - Environment Variable Groups 49**

**E. Appendix E - Error and Safety Contract 50**

E.1 Representative stable codes . . . . . 50

**F. Appendix F - Verification Record 51**

**G. Appendix G - References 52**

**H. Appendix H - Glossary 53**

**End of Report 54**

# Lists of Figures and Tables

---

- Figure 1. Cipherboard runtime boundaries
  - Figure 2. Repository source distribution by line count
  - Figure 3. Allowed game lifecycle transitions
  - Figure 4. Authoritative attempt transaction
  - Figure 5. Private duel event flow
  - Figure 6. Security boundaries and control layers
  - Figure 7. Privacy lifecycle for account and gameplay data
  - Figure 8. Provider-neutral production topology
  - Figure 9. Automated verification results
  - Figure 10. Measured coverage evidence
- 
- Table 1. Product stakeholders and needs
  - Table 2. Requirement traceability summary
  - Table 3. Official game presets
  - Table 4. Runtime responsibilities
  - Table 5. Primary API areas
  - Table 6. Principal data entities
  - Table 7. Threat-control summary
  - Table 8. Verification evidence
  - Table 9. Production readiness assessment
  - Table 10. Prioritized recommendations

# 1. Executive Summary

I developed Cipherboard as a complete logic-game platform rather than a browser-only puzzle. A new player receives an anonymous Supabase identity and can start in under a minute. The API owns the secret, validates each guess, calculates feedback, records attempts, computes score, awards achievements, and decides multiplayer results. The browser presents the experience but cannot create an accepted competitive outcome by changing local state.

The repository uses a pnpm and uv monorepo. The main runtime components are the Next.js web application, FastAPI service, pure Python game engine, PostgreSQL database, Redis coordination layer, Supabase Auth integration, generated TypeScript API contract, and the original SBA command-line application. Production support includes Docker images, Alembic migrations, release scripts, operational runbooks, observability definitions, privacy schedules, and security guidance.

## My system objective

I wanted one set of rules, a fast route into play, strong competitive integrity, and enough operational discipline to run the product beyond a classroom demonstration.

## 1.1 Key outcomes

- **Correctness:** one canonical engine calculates duplicate-safe feedback and legal state transitions.
- **Integrity:** PostgreSQL transactions, row locks, unique constraints, idempotency keys, and server timestamps protect official outcomes.
- **Privacy:** active secrets use AES-256-GCM, invite capabilities are stored as digests, and analytics excludes identity and gameplay secrets.
- **Access:** keyboard support, non-colour symbols, screen-reader announcements, reduced motion, 44px controls, and two locales form part of the product contract.
- **Operations:** health checks, metrics, migrations, backup guidance, rollback, retention, key rotation, and incident playbooks support controlled deployment.

## 1.2 Readiness judgment

The codebase shows a strong production-oriented design and passed the local static and unit checks that I ran. I would not approve a public launch from this evidence alone. I still need results from the pinned Node runtime, PostgreSQL integration services, browser end-to-end suites, accessibility review, container scans, load tests, restore drills, legal configuration, and live managed infrastructure. This judgment follows the repository launch checklist and avoids treating design documents as proof of deployment.

## 2. Methodology and Evidence

I inspected product, architecture, security, privacy, operations, source, migration, and test files. I traced important behavior from a requirement to code and then to a verification path. I treated repository documentation as design evidence and source code as implementation evidence. Test output supplies verification evidence for the commands I ran.

### 2.1 Evidence hierarchy

| Priority | Evidence                                | How I used it                                                  |
|----------|-----------------------------------------|----------------------------------------------------------------|
| 1        | User objective and product requirements | Defined the required outcome and report scope.                 |
| 2        | Source code and migrations              | Established implemented runtime behavior and data constraints. |
| 3        | Automated test output                   | Established observed behavior in the local environment.        |
| 4        | Architecture and operations documents   | Explained intent, assumptions, and operator procedures.        |
| 5        | Targets and checklists                  | Identified work that still needs measured release evidence.    |

### 2.2 Commands executed for this report

| Command                                                                         | Result                                                     |
|---------------------------------------------------------------------------------|------------------------------------------------------------|
| uv run pytest                                                                   | 133 passed; 8 skipped; 141 collected.                      |
| uv run pytest tests/core --cov=mastermind_core --cov-branch --cov-fail-under=95 | 42 passed; 100% measured statement and branch coverage.    |
| uv run ruff check .                                                             | Passed.                                                    |
| pnpm typecheck                                                                  | Passed with Node engine warning.                           |
| pnpm test                                                                       | 47 tests passed across shared config, API client, and web. |

### 2.3 Assessment limits

I did not run production migrations, public deployment, browser end-to-end tests, load tests, container builds, vulnerability scanners, restore operations, or key-rotation drills for this report. Eight Python integration cases skipped because their PostgreSQL conditions were not active. The local Node version was 20.20.0, while the repository requires Node 24.18.0. I identify those gaps where they affect my conclusion.

### 3. Product and Stakeholder Context

Cipherboard serves casual puzzle players, students practising logic, friends sharing challenges, competitive daily players, teachers, support staff, administrators, and operators. I used anonymous authenticated identity to remove the account-registration barrier without replacing identity with an untrusted browser identifier.

Table 1. Product stakeholders and needs

| Stakeholder        | Need                                       | System response                                                |
|--------------------|--------------------------------------------|----------------------------------------------------------------|
| New player         | Begin quickly and understand the board.    | Anonymous sign-in, presets, guide, clear next action.          |
| Student            | Practise reasoning and review results.     | Stable symbols, history, score breakdown, CLI learning path.   |
| Friend group       | Share private asynchronous and live games. | High-entropy challenge links and two-player rooms.             |
| Competitive player | Trust rankings and daily fairness.         | Server score, official configuration, stable ties, moderation. |
| Teacher            | Demonstrate algorithms and data handling.  | Canonical Python engine and SBA CLI documentation.             |
| Operator           | Deploy, observe, recover, and audit.       | Health checks, metrics, runbooks, migrations, audit events.    |

#### 3.1 Product principles

I aimed for a calm, precise, and quietly playful interface. Players should focus on logical deduction, so the product avoids casino styling, neon effects, manipulative streak pressure, advertising trackers, fake metrics, and colour-only feedback. The design targets users aged 13 and above and does not collect date of birth.

#### 3.2 Supported experiences

The product supports Easy, Normal, Hard, Expert, Custom, Practice, pass-and-play, daily, friend challenge, and private duel modes. Linked accounts add cross-device history, moderated display names, achievements, public leaderboard opt-in, export, and deletion without discarding the guest identity history during upgrade.

## 4. Requirements and Traceability

I converted the product goal into functional requirements and quality attributes. I then traced each requirement to a component and verification source. The complete route, entity, environment, and source catalogues appear in the appendices.

Table 2. Requirement traceability summary

| ID     | Requirement                      | Implementation                                         | Evidence                         |
|--------|----------------------------------|--------------------------------------------------------|----------------------------------|
| FR-01  | Authenticated guest play         | Session provider, Supabase Auth, JWT principal         | Auth and product-flow tests      |
| FR-02  | Correct duplicate-safe feedback  | mastermind_core.feedback                               | Core feedback tests              |
| FR-03  | One official daily run           | Daily challenge and unique owner constraint            | Daily API tests                  |
| FR-04  | Private friend capability        | Random share token, digest, expiry, revocation         | Friend-room tests                |
| FR-05  | Recoverable real-time duel       | Tickets, sequence events, replay, PostgreSQL authority | Realtime and room tests          |
| FR-06  | Account export and deletion      | Export archive and deletion saga                       | Profile/admin/support tests      |
| NFR-01 | WCAG 2.2 AA target               | Keyboard, symbols, live regions, reduced motion        | A11y suite defined; not run here |
| NFR-02 | Server-authoritative competition | API transactions and score_v1                          | Game, achievement, schema tests  |
| NFR-03 | Privacy-limited analytics        | Closed event schema and consent flags                  | Analytics tests                  |
| NFR-04 | Operational recoverability       | Backups, rollback, incident and key runbooks           | Document review; drills pending  |

### 4.1 Official preset rules

Table 3. Official game presets

| Preset | Colours | Code length | Attempts | Duplicates | Ranked |
|--------|---------|-------------|----------|------------|--------|
| Easy   | 5       | 4           | 12       | No         | Yes    |
| Normal | 6       | 4           | 10       | Yes        | Yes    |
| Hard   | 8       | 5           | 8        | Yes        | Yes    |
| Expert | 10      | 6           | 8        | Yes        | Yes    |

Custom and practice configurations remain unranked by default. The validation bounds are 5 to 10 colours, code length 3 to 6, and 1 to 20 attempts. A configuration without duplicates must provide at least as many colours as code positions.

## 5. System Architecture

I separated Cipherboard into presentation, delivery, domain, persistence, and coordination layers. The browser renders the board and keeps the unsubmitted row. FastAPI authenticates requests, authorizes resources, and executes transactions. The pure Python engine owns rules. PostgreSQL stores authoritative outcomes. Redis coordinates rate limits, presence, one-use WebSocket tickets, and fan-out, but its loss cannot rewrite a completed game.

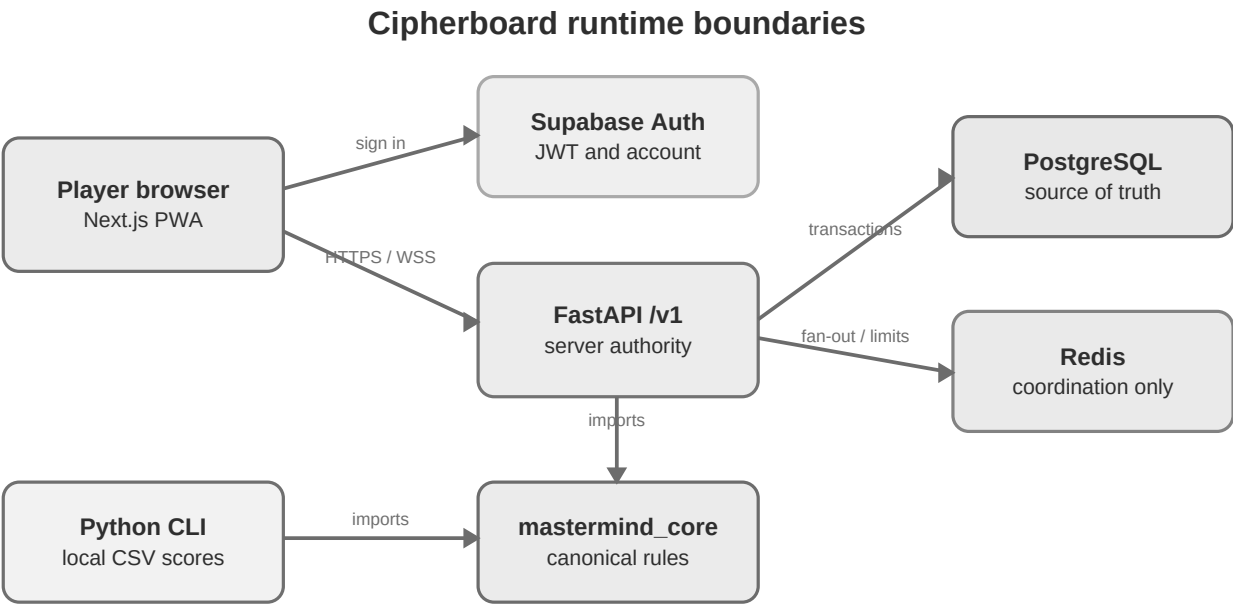


Figure 1. Cipherboard runtime boundaries. Arrows show the major authentication, request, persistence, and shared-engine paths.

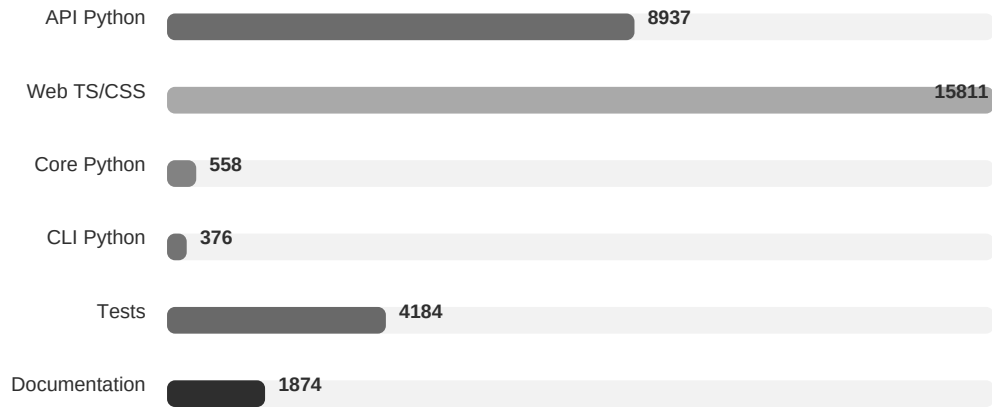
Table 4. Runtime responsibilities

| Component       | Responsibility                                               | Excluded responsibility                   |
|-----------------|--------------------------------------------------------------|-------------------------------------------|
| Next.js PWA     | Interaction, rendering, localization, safe local draft state | Accepted feedback, score, opponent secret |
| FastAPI         | Auth, authorization, transactions, response contracts        | Presentation and local UI preferences     |
| mastermind_core | Rules, validation, feedback, scoring, daily derivation       | Database, HTTP, browser, framework state  |
| PostgreSQL      | Games, attempts, rooms, events, ranks, audits                | Ephemeral connection fan-out              |
| Redis           | Limits, presence, tickets, cache, horizontal fan-out         | Authoritative game outcomes               |
| Supabase Auth   | Anonymous and linked identity, JWT issuance                  | Game authorization decisions              |
| Python CLI      | Local educational interface and CSV scores                   | Networked ranked competition              |

### 5.1 Repository scale

I counted source lines by component to show where implementation and documentation effort sits. The counts include blank and comment lines and serve as a structural view, not a productivity measure.

### Repository lines by component



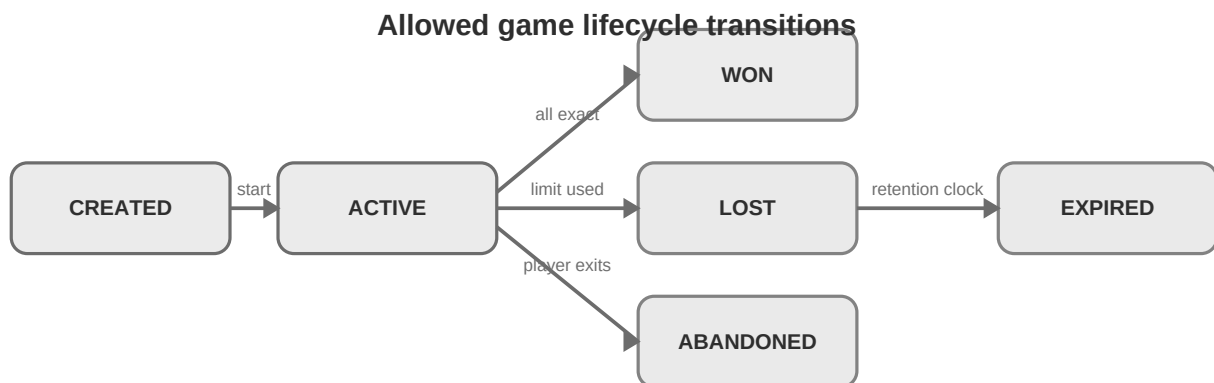
*Figure 2. Repository line counts calculated from tracked text files in the current workspace.*

## 5.2 Version boundaries

The public API uses **v1**. Persisted sessions record **rules\_v1**, **score\_v1**, and the applicable daily derivation version. I used those explicit versions so later algorithm changes do not make old official games impossible to reproduce.

## 6. Mastermind Domain Model

A game configuration defines a stable colour palette, code length, maximum attempts, duplicate rule, Code Maker type, and ranking setting. The engine normalizes input before it changes state. An invalid guess raises a domain error and consumes no attempt, time record, score, or database row.



*Figure 3. The engine allows terminal transitions from active play and prevents later guesses from mutating a completed game.*

### 6.1 Feedback calculation

I calculate exact-position matches first. I remove those pegs from both sequences, count the remaining colours, and sum the intersection of the two counters. This method prevents a duplicate colour from earning more feedback pegs than either the secret or guess contains. Its time complexity is  $O(n)$  for a code of length  $n$ .

**Worked example:** secret A A B C and guess A B A A produce one exact match at position 1. The unmatched secret contains A, B, C and the unmatched guess contains B, A, A. Counter intersection contributes A once and B once, so the result is 1 exact and 2 misplaced.

### 6.2 Scoring model

The versioned score rewards harder configurations, fewer attempts, and bounded completion time. Only server-observed configuration, status, attempts, and elapsed time feed the calculation. The client never submits a final score. A leaderboard entry also stores attempts and elapsed time so the API can apply deterministic tie-breaks.

## 7. Canonical Game Engine

I placed all rule logic in **packages/mastermind\_core**. The package has no FastAPI, SQLAlchemy, Redis, browser, or terminal dependency. Both the API and CLI import it, so a feedback correction changes one implementation rather than two competing rule sets.

### 7.1 Duplicate-safe feedback implementation

```
1 from __future__ import annotations
2
3 from collections import Counter
4 from collections.abc import Sequence
5
6 from .errors import DomainError
7 from .models import Feedback
8
9
10 def calculate_feedback(secret: Sequence[str], guess: Sequence[str]) -> Feedback:
11     if len(secret) != len(guess):
12         raise DomainError("SEQUENCE_LENGTH_MISMATCH", "Secret and guess lengths must match.")
13     black = 0
14     unmatched_secret: list[str] = []
15     unmatched_guess: list[str] = []
16     for secret_peg, guess_peg in zip(secret, guess, strict=True):
17         if secret_peg == guess_peg:
18             black += 1
19         else:
20             unmatched_secret.append(secret_peg)
21             unmatched_guess.append(guess_peg)
22     secret_counts = Counter(unmatched_secret)
23     guess_counts = Counter(unmatched_guess)
24     white = sum((secret_counts & guess_counts).values())
25     return Feedback(black=black, white=white)
```

*Code example 1. Actual source from feedback.py. Exact matches are removed before Counter intersection.*

## 7.2 Immutable guess transition

```

34 def submit_guess(
35     state: GameState,
36     secret: Sequence[str],
37     guess: str | Sequence[str],
38     *,
39     at: datetime | None = None,
40     idempotency_key: str | None = None,
41 ) -> GameState:
42     if state.status is not GameState.ACTIVE:
43         raise DomainError("GAME_NOT_ACTIVE", "This game no longer accepts attempts.")
44     normalized_secret = validate_code(secret, state.config)
45     normalized_guess = validate_code(guess, state.config)
46     timestamp = at or datetime.now(UTC)
47     feedback = calculate_feedback(normalized_secret, normalized_guess)
48     attempt = AttemptRecord(
49         number=state.attempts_used + 1,
50         guess=normalized_guess,
51         feedback=feedback,
52         submitted_at=timestamp,
53         idempotency_key=idempotency_key,
54     )
55     attempts = (*state.attempts, attempt)
56     target = (
57         GameState.WON
58         if feedback.black == state.config.code_length
59         else GameState.LOST
60         if len(attempts) >= state.config.max_attempts
61         else GameState.ACTIVE
62     )
63     completed_at = timestamp if target.terminal else None
64     elapsed = max(0.0, (timestamp - (state.started_at or timestamp)).total_seconds())
65     score = (
66         calculate_score(
67             state.config,
68             status=target,
69             attempts_used=len(attempts),
70             elapsed_seconds=elapsed,
71         )
72         if target.terminal
73         else None
74     )
75     return replace(
76         state,
77         status=target,
78         attempts=attempts,
79         completed_at=completed_at,
80         score=score,
81     )

```

*Code example 2. Actual source from engine.py. A valid guess creates a new attempt tuple and derives the terminal state and score.*

## 7.3 Design evaluation

The pure engine gives me deterministic unit tests and a narrow security boundary. The API remains responsible for encryption, persistence, authorization, and concurrency. This division avoids mixing business rules with transport or database failure handling. The protocol-based random source also lets tests inject deterministic generation without weakening production randomness.

## 8. Game Modes and User Journeys

### 8.1 Solo, practice, and custom play

A player selects an official preset or a valid custom configuration. The API creates an active game and returns a public game identifier without the secret. The board submits one normalized row at a time. Results expose the permitted secret, score breakdown, achievements, and rank only after a terminal state.

### 8.2 Daily challenge

The server derives one puzzle for each UTC date and derivation version. The first valid guess begins the official run. A unique owner-and-daily constraint prevents a second official outcome, including during concurrent starts. Refresh restores the same session. Later plays appear as unranked practice, and a run that crosses midnight stays attached to its start date.

### 8.3 Friend challenge

The creator chooses rules, an optional bounded title, name visibility, expiry, and a generated or human secret. The server returns a high-entropy share capability once and stores only its digest. Each solver receives one official outcome. The creator can revoke the challenge and view aggregate results without player identifiers, names, guess histories, or secrets.

### 8.4 Private duel

Two authorized room members receive the same server-generated code and submit guesses independently. The opponent sees presence, attempt count, and terminal progress, not private guesses. The API uses server receipt time and a configured tie window to decide the outcome. Durable sequence numbers allow replay after reconnect without duplicating effects.

### 8.5 Failure states

The web application includes loading, error, offline, degraded, empty, and reconnecting states. Ranked and race-sensitive mutations fail closed when Redis coordination is unavailable. Ordinary analytics failure does not block gameplay. Database or unverifiable-auth failure rejects protected mutations because accepting them would create outcomes the server cannot trust.

## 9. Web Application

The web client uses strict TypeScript and the Next.js App Router. Locale-prefixed routes cover play, games, results, daily, challenges, rooms, profile, achievements, leaderboards, account controls, support, administration, accessibility, privacy, terms, offline, and degraded states.

### 9.1 Client boundaries

The browser may preserve the current unsubmitted row and user preferences. It does not calculate accepted feedback or score. The session provider obtains a real anonymous Supabase identity, upgrades the same identity for registered users, and sends the access token to the API. Public API calls use explicit bearer authentication rather than ambient game cookies.

### 9.2 Game board implementation

```

79 export function GameBoard({
80   title,
81   intro,
82   startEndpoint = '/v1/games',
83   startBody = { mode: 'solo', difficulty: 'normal' },
84   replayBody,
85   replayLabel,
86   dailyChallengeId,
87   initialGameId,
88 }: GameBoardProps) {
89   const t = useTranslations('Game');
90   const tr = useTranslations('Result');
91   const ta = useTranslations('Achievements');
92   const api = useApi();
93   const analytics = useProductAnalytics();
94   const [game, setGame] = useState<Game | null>(null);
95   const [row, setRow] = useState<string[]>([]);
96   const [selectedSlot, setSelectedSlot] = useState(0);
97   const [state, setState] = useState<'idle' | 'loading' | 'ready' | 'submitting' | 'error'>(
98     initialGameId || dailyChallengeId ? 'loading' : 'idle',
99   );
100   const [error, setError] = useState<string | null>(null);
101   const [online, setOnline] = useState(true);
102   const [syncing, setSyncing] = useState(false);
103   const [syncRequired, setSyncRequired] = useState(false);
104   const [announcement, setAnnouncement] = useState('');
105   const [elapsed, setElapsed] = useState(0);
106   const [resultDetails, setResultDetails] = useState<ResultDetails | null>(null);
107   const slotRefs = useRef<Array<HTMLButtonElement | null>>([]);
108   const resultRef = useRef<HTMLDivElement | null>(null);
109   const achievementBaselineRef = useRef<Set<string> | null>(null);
110   const achievementBaselineLoadingRef = useRef(false);
111   const gameRef = useRef<Game | null>(null);
112   const rowRef = useRef<string[]>([]);
113
114   const hydrateGame = useCallback(
115     (nextGame: Game) => {
116       setResultDetails(null);
117       gameRef.current = nextGame;
118       setGame(nextGame);
119       if (dailyChallengeId) {
120         const storageKey = dailyGameStorageKey(dailyChallengeId);
121         if (nextGame.mode === 'daily' && nextGame.status === 'active')
122           localStorage.setItem(storageKey, nextGame.id);
123         else localStorage.removeItem(storageKey);
124       }
125       if (
126         nextGame.status === 'active' &&
127         achievementBaselineRef.current === null &&
128         !achievementBaselineLoadingRef.current
129       ) {
130         achievementBaselineLoadingRef.current = true;
131         void api<Stats>('/v1/me/stats')
132           .then((stats) => {
133             achievementBaselineRef.current = new Set(stats.achievements ?? []);
134           })
135           .catch(() => undefined)
136           .finally(() => {
137             achievementBaselineLoadingRef.current = false;
138           });
139       }
140       setRow((current) => {
141         const storageKey = rowStorageKey(nextGame.id);
142         if (current.length > 0) return current;

```

*Code example 3. Actual React component source showing the typed GameBoard interface and local interaction state.*

### 9.3 Responsive interaction

The board supports pointer and keyboard input, colour-independent patterns, focus visibility, 44-pixel targets, live announcements, and reduced motion. Mobile end-to-end specifications exist for narrow layouts and gameplay. The visual language uses restrained spacing and stable peg geometry so feedback remains legible without casino or worksheet styling.

## 10. Progressive Web Application and Resilience

Cipherboard includes a web manifest, install icons, a service worker, and an offline route. I describe the offline state honestly: cached application assets may load, but authoritative gameplay needs the API. The client can preserve a safe unsubmitted row and resynchronize the game after connection returns.

### 10.1 Cache and update boundary

The service worker must not treat authenticated API responses as durable offline truth. API responses under /v1 receive no-store behavior, while static assets support installation and shell loading. This boundary protects secret omission and prevents a stale browser cache from presenting an old competitive outcome as current.

### 10.2 Degraded service

The dedicated degraded page gives the player a clear explanation when a dependency blocks safe gameplay. Redis degradation may still allow non-sensitive reads while ranked, invite, room, and race-sensitive writes fail closed. This user-facing distinction matches the backend readiness model.

# 11. API and Backend Services

FastAPI exposes a versioned JSON API and one WebSocket route. Pydantic models form explicit request and response allowlists. The service never serializes ORM objects as public responses, which reduces the chance that ciphertext, internal identifiers, moderation fields, or operator data appears by accident.

Table 5. Primary API areas

| Area           | Representative operations                                  |
|----------------|------------------------------------------------------------|
| Games          | Create, restore, submit attempt, abandon                   |
| Daily          | Definition, official start, leaderboard                    |
| Challenges     | Create, manage, resolve capability, start, results, revoke |
| Rooms          | Create, join, ready, state, WebSocket ticket, events       |
| Player         | Profile, stats, history, export, delete                    |
| Competition    | Public leaderboard and achievements                        |
| Operations     | Live, ready, metrics, support, analytics                   |
| Administration | Review, moderation, flags, terminate, audit                |

## 11.1 Request safety

Middleware enforces a body limit, timeout, exact CORS policy, request and correlation identifiers, per-IP and per-subject rate limits, no-store API caching, and security headers. Errors use a stable code, safe message, and request ID. Internal exceptions and encrypted data do not enter the response.

## 11.2 Idempotent creation

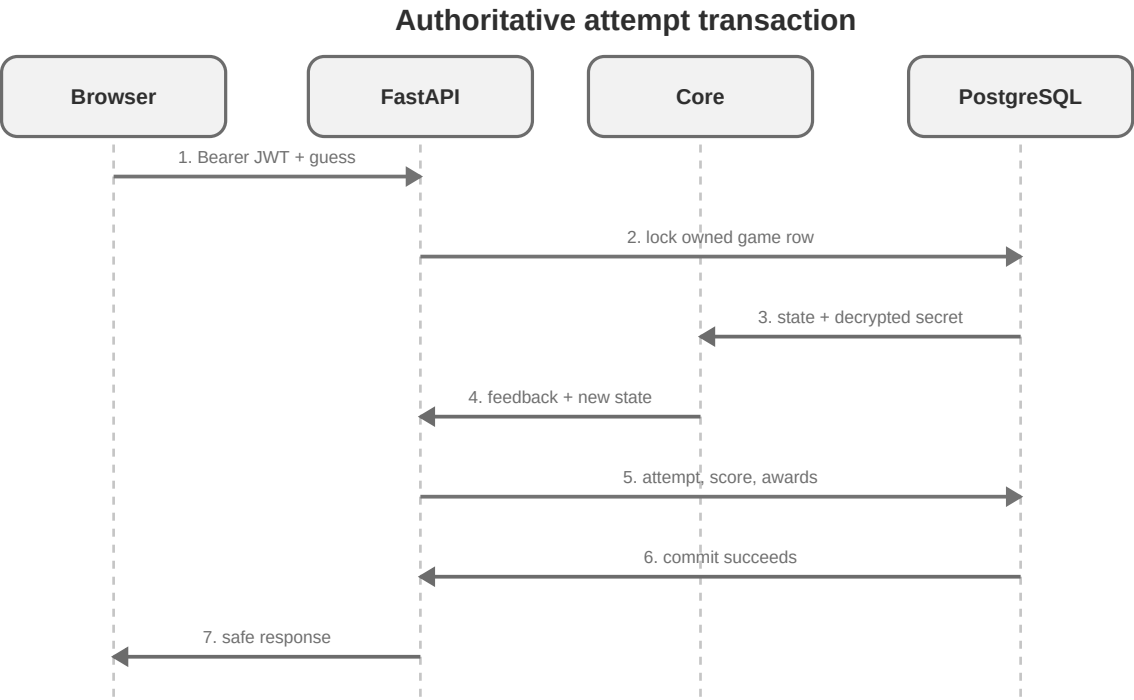
```

319 async def create_game(
320     session: AsyncSession,
321     principal: AuthPrincipal,
322     request: CreateGameRequest,
323     cipher: SecretCipher,
324     *,
325     secret_override: tuple[str, ...] | None = None,
326     daily_id: uuid.UUID | None = None,
327     friend_id: uuid.UUID | None = None,
328     room_id: uuid.UUID | None = None,
329     linked_expires_at: datetime | None = None,
330     commit: bool = True,
331     settings: Settings | None = None,
332 ) -> GameSession:
333     await ensure_profile(session, principal)
334     fingerprint = None
335     if request.idempotency_key:
336         fingerprint = creation_request_fingerprint(
337             "game.create",
338             request.model_dump(mode="json", by_alias=True, exclude={"idempotency_key"})
339             | {
340                 "dailyId": str(daily_id) if daily_id else None,
341                 "friendId": str(friend_id) if friend_id else None,
342                 "roomId": str(room_id) if room_id else None,
343             },
344             settings or get_settings(),
345         )
346     existing = await session.scalar(
347         select(GameSession)
348         .options(selectinload(GameSession.attempts))
349         .where(
350             GameSession.owner_id == principal.user_id,
351             GameSession.creation_idempotency_key == request.idempotency_key,
352         )
353     )
354     if existing:
355         if (
356             existing.creation_request_fingerprint is not None
357             and existing.creation_request_fingerprint != fingerprint
358         ):
359             raise _idempotency_reused()
360         _assert_game_creation_matches(
361             existing,
362             request,
363             cipher,
364             daily_id=daily_id,
365             friend_id=friend_id,
366             room_id=room_id,
367         )
368         return existing
369     if request.mode in {GameMode.DAILY, GameMode.FRIEND_CHALLENGE, GameMode.DUEL} and not (
370         daily_id or friend_id or room_id
371     ):
372         raise APIError(
373             400, "DEDICATED_ENDPOINT_REQUIRED", "Use the dedicated endpoint for this mode."
374         )
375     config, difficulty = _expected_game_config(request)
376     if (
377         config.code_maker is CodeMakerType.HUMAN
378         and request.mode is not GameMode.PASS_AND_PLAY
379         and friend_id is None
380         and secret_override is None
381     ):
382         raise APIError(400, "INVALID_CODE_MAKER", "Human Code Maker is available in pass-and-play.")
383     if request.secret is not None:
384         if config.code_maker is not CodeMakerType.HUMAN and not friend_id:
385             raise APIError(
386                 400, "SECRET_NOT_ACCEPTED", "A secret is not accepted for this game mode."
387             )
388         secret_code = validate_code(request.secret, config)
389     else:
390         secret_code = secret_override or generate_secret(config)
391     public_id = public_token()
392     encrypted = cipher.encrypt(secret_code, context_cipher_context(public_id))
393     started_at = utcnow()

```

**Code example 4.** Actual service code checks request fingerprints and dedicated mode boundaries before generating and encrypting a secret.

### 11.3 Attempt transaction



**Figure 4.** The API locks the owned game, calls the canonical engine, writes immutable results, and publishes only after commit.

## 12. Identity, Authentication, and Accounts

Every player, including a guest, has a cryptographically verified Supabase subject. The API accepts a bearer token, retrieves the signing key from the issuer JWKS, allowlists RS256 and ES256, and checks issuer, audience, expiry, issued time, and subject. It converts the subject to a UUID before constructing the principal.

```

34 class SupabaseJWTVerifier:
35     def __init__(self, settings: Settings) -> None:
36         if not settings.supabase_url:
37             raise RuntimeError("MASTERMIND_SUPABASE_URL is required for authentication.")
38         self.issuer = f"{settings.supabase_url.rstrip('/')}/auth/v1"
39         self.audience = settings.supabase_jwt_audience
40         self.jwks = PyJWKClient(
41             f"{self.issuer}/.well-known/jwks.json",
42             cache_jwk_set=True,
43             lifespan=600,
44             cache_keys=True,
45             max_cached_keys=16,
46         )
47
48     def verify(self, token: str) -> AuthPrincipal:
49         try:
50             signing_key = self.jwks.get_signing_key_from_jwt(token)
51             claims: dict[str, Any] = jwt.decode(
52                 token,
53                 signing_key.key,
54                 algorithms=["RS256", "ES256"],
55                 audience=self.audience,
56                 issuer=self.issuer,
57                 leeway=JWT_CLOCK_SKEW_SECONDS,
58                 options={"require": ["exp", "iat", "iss", "aud", "sub"]},
59             )
60             user_id = uuid.UUID(str(claims["sub"]))
61             authentication_times = [
62                 int(item["timestamp"])
63                 for item in claims.get("amr", [])
64                 if isinstance(item, dict) and isinstance(item.get("timestamp"), int | float)
65             ]
66             authenticated_at = (
67                 datetime.fromtimestamp(max(authentication_times), tz=UTC)
68                 if authentication_times
69                 else None
70             )
71             return AuthPrincipal(
72                 user_id=user_id,
73                 is_anonymous=bool(claims.get("is_anonymous", False)),
74                 session_id=claims.get("session_id")
75                 if isinstance(claims.get("session_id"), str)
76                 else None,
77                 issued_at=datetime.fromtimestamp(int(claims["iat"]), tz=UTC),
78                 authenticated_at=authenticated_at,
79                 assurance_level=(claims.get("aal") if isinstance(claims.get("aal"), str) else None),
80             )
81         except (InvalidTokenError, ValueError, KeyError) as exc:
82             raise APIError(
83                 401, "INVALID_ACCESS_TOKEN", "Your session is invalid or has expired."
84             ) from exc

```

*Code example 5. Actual JWT verification source with required claims, issuer, audience, algorithm allowlist, and bounded clock skew.*

### 12.1 Authorization

Knowledge of a UUID or share code does not grant access to a private object. Database queries include the owner or room-member predicate. Administrative access adds a permanent account, server-side grant, AAL2, recent authentication, active profile, and non-revoked grant. The route denies access if any condition fails.

### 12.2 Account lifecycle

A guest can link the same identity to email or another supported provider. Registered users can export application data as a protected ZIP archive. Deletion requires the exact DELETE confirmation and a recent authenticated session. The system hides and anonymizes application data, revokes the presented session, deletes the Auth identity, and records a retry request if the external provider step fails.

## 13. Data Architecture and Persistence

PostgreSQL is the source of truth for games, attempts, daily definitions, challenges, rooms, multiplayer events, leaderboard rows, achievements, flags, moderation, audits, support, deletion requests, and product events. Alembic is the sole schema authority. The API does not create tables during startup.

*Table 6. Principal data entities*

| Entity                   | Purpose                                         | Key integrity control                              |
|--------------------------|-------------------------------------------------|----------------------------------------------------|
| profiles                 | Identity projection and public preferences      | UUID primary key; normalized name uniqueness       |
| game_sessions            | Authoritative lifecycle, versions, score, links | State checks; unique official and idempotency keys |
| game_attempts            | Immutable guesses and feedback                  | Unique attempt number and idempotency per game     |
| daily_challenges         | UTC definition and derivation versions          | Unique date and rule-set version                   |
| friend_challenges        | Private share capability and encrypted secret   | Unique token digest; creator quotas                |
| multiplayer_rooms/events | Duel state and durable replay                   | Unique room sequence; allowed status checks        |
| leaderboard_entries      | Stable ranked outcome                           | One row per game; bounded score fields             |
| audit_events             | Administrative accountability                   | Actor, action, target, reason, timestamp           |

### 13.1 Concurrency and invariants

The attempt path locks the game row before checking state and appending an attempt. Unique constraints protect concurrent duplicates after application checks. The schema restricts attempt counts, allowed modes and statuses, score ranges, feedback totals, room event sequences, and one official run per owner and daily, friend challenge, or room.

### 13.2 Redis boundary

Redis supports counters, presence leases, fan-out, cache, and one-use tickets. The product can rebuild safe real-time state from PostgreSQL sequence events. Redis loss may reduce availability, but it must not change a stored winner or accepted attempt.

## 14. Scoring, Rankings, and Achievements

The API creates scores for terminal official games using `score_v1`. Ranked eligibility requires an official game mode, an exact official preset, a supported difficulty, and an eligible outcome. Custom rules cannot enter an official difficulty board by copying a label.

### 14.1 Deterministic order

Leaderboard rows sort by score descending, attempts ascending, validated elapsed time ascending, completion timestamp ascending, and immutable row identifier. The last key stabilizes pagination when two players share all visible values. Moderation state can remove or restore an entry without rewriting the underlying game.

### 14.2 Achievements

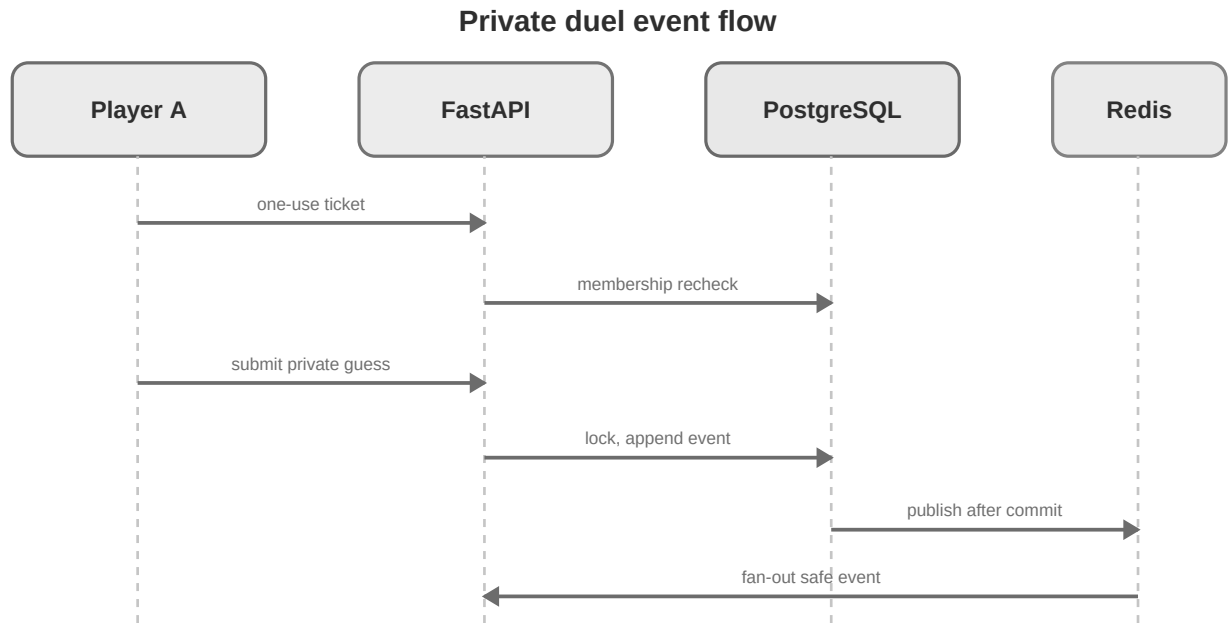
Launch achievements include First Break, One Shot, No Waste, Daily Debut, Logic Week, Hard Mode, Expert Breaker, Challenger, and Duelist. The server awards them after authoritative outcomes and uses a unique user-and-achievement constraint to make the operation idempotent.

### 14.3 Anti-cheat position

A client submits guesses, not scores or elapsed totals. The server owns start time, completion time, configuration, secret, feedback, attempt count, and rank eligibility. Review status and audit-backed invalidation provide a controlled response to anomalies without allowing silent score edits.

## 15. Real-Time Multiplayer

Private duels require both players to receive the same secret and independent feedback without learning the guesses submitted by the other player. I used WebSockets for timely updates and PostgreSQL events for durable replay. Redis distributes safe events between API instances.



**Figure 5.** A room member authenticates with a one-use ticket. The API commits the authoritative event before Redis fan-out.

## 15.1 Connection security

```

548 @router.websocket("/{room_id}/events")
549 async def room_events(websocket: WebSocket, room_id: uuid.UUID) -> None:
550     settings = get_settings()
551     if not settings.feature_multiplayer:
552         await websocket.close(code=1013, reason="Private duels are unavailable.")
553     return
554     ticket = _websocket_ticket(websocket)
555     if not ticket:
556         await websocket.close(code=4401, reason="Authentication required.")
557     return
558     try:
559         redis = await _ensure_realtime_available(
560             websocket app state, settings, require_ticket_store=True
561         )
562     except APIError:
563         await websocket.close(code=1013, reason="Realtime authentication is unavailable.")
564     return
565     assert redis is not None
566     try:
567         ticket_hash = hashlib.sha256(ticket.encode()).hexdigest()
568         async with asyncio.timeout(2):
569             stored = await redis.getdel(f"mastermind:ws-ticket:{ticket_hash}")
570         ticket_data = json.loads(stored) if stored else None
571         if (
572             not isinstance(ticket_data, dict)
573             or ticket_data.get("roomId") != str(room_id)
574             or datetime.fromisoformat(str(ticket_data.get("expiresAt"))) <= utcnow()
575         ):
576             raise ValueError("invalid ticket")
577         principal = AuthPrincipal(
578             user_id=uuid.UUID(str(ticket_data["userId"])),
579             is_anonymous=bool(ticket_data.get("isAnonymous", True)),
580             session_id=None,
581             issued_at=utcnow(),
582             authenticated_at=None,
583             assurance_level=None,
584         )
585     except (RedisError, TimeoutError):
586         await websocket.close(code=1013, reason="Realtime authentication is unavailable.")
587     return
588     except (ValueError, TypeError, json.JSONDecodeError):
589         await websocket.close(code=4401, reason="Invalid or expired realtime ticket.")
590     return
591     broker = Broker = websocket.app.state.broker
592     if not broker.available:
593         await websocket.close(code=1013, reason="Realtime service unavailable.")
594     return
595     async with SessionFactory() as session:
596         if not await feature_enabled(session, settings, "multiplayer"):
597             await websocket.close(code=1013, reason="Private duels are unavailable.")
598         return
599         try:
600             room = await load_room_for_member(session, room_id, principal)
601         except APIError:
602             await websocket.close(code=4403, reason="Room access denied.")
603             return
604         if room.status == "expired":
605             await websocket.close(code=4408, reason="Room expired.")
606             return
607         member_exists = await session.scalar(
608             select(MultiplayerMember) where(
609                 MultiplayerMember.room_id == room_id,
610                 MultiplayerMember.user_id == principal.user_id,
611             )
612         )
613         if member_exists is None:

```

*Code example 6. Actual WebSocket route source showing bounded ticket handling, origin checks, and membership flow.*

## 15.2 Winner and tie logic

The database transaction records the first winning server timestamp. A bounded tie window allows a second result to become a tie when receipt times fall close enough. Client clocks cannot decide the result. Sequence numbers and idempotent commands prevent a reconnect or retry from applying the same effect twice.

## 16. Security Architecture

I treat browser identifiers, display names, timestamps, idempotency keys, forwarded headers, and WebSocket commands as hostile input. The API creates trust only after authentication, schema validation, authorization, and transaction checks.

Security boundaries and control layers

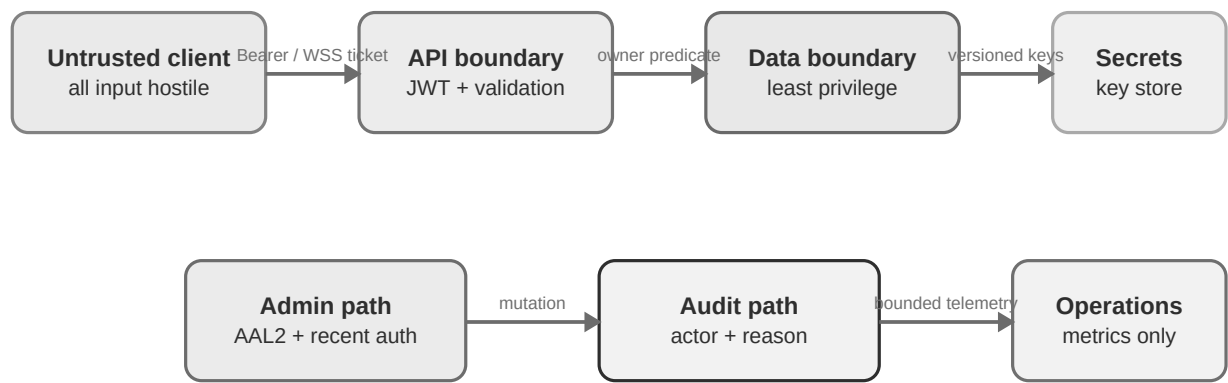


Figure 6. Security controls accumulate at the client, API, data, secret, administrative, audit, and operations boundaries.

Table 7. Threat-control summary

| Threat             | Primary control                                                   | Verification or detection                  |
|--------------------|-------------------------------------------------------------------|--------------------------------------------|
| Secret disclosure  | AES-256-GCM, explicit DTOs, redaction, terminal reveal rules      | Serialization and log scanning tests       |
| Forged JWT         | JWKS signature, exact issuer/audience, claim and algorithm checks | Invalid-token test matrix and auth metrics |
| IDOR/BOLA          | Owner/member predicates and private serializers                   | Authorization integration cases            |
| Score manipulation | Server rules, row locks, unique official outcomes                 | Concurrency and game tests                 |
| Invite enumeration | 128+ bit capability, digest storage, generic not-found            | Invalid-token and limit metrics            |
| WebSocket abuse    | One-use ticket, frame and connection limits, heartbeat            | Close reasons and socket metrics           |
| Admin escalation   | Server grant, AAL2, recent auth, audit                            | Admin auth failures and audit review       |
| Key loss           | External versioned key ring and fail-closed decryption            | Readiness and rotation drills              |

### 16.1 Secret protection

Standard game and human challenge secrets use AES-256-GCM with a nonce, ciphertext, context, and key version. Daily secrets derive from independent versioned HMAC material. Friend and room capability tokens use an independent digest key. The system does not reuse AES material for HMAC purposes.

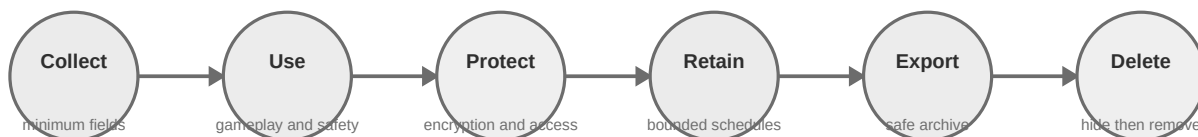
### 16.2 Residual assumptions

Production security depends on modern TLS ingress, private managed data services, protected CI environments, MFA, secret management, backups, and disciplined operators. The launch checklist blocks release if these assumptions lack evidence or compensating controls.

## 17. Privacy and Data Governance

I collect data needed for identity, gameplay, ranked integrity, social features, moderation, and operations. The product excludes advertising trackers, date of birth, required real names, secret codes in analytics, full guess histories in third-party reporting, and browser exposure of server credentials.

### Privacy lifecycle for account and gameplay data



*Active secrets and invite capabilities never enter analytics, logs, URLs, or exports.*

**Figure 7.** The data lifecycle separates collection, use, protection, retention, export, and deletion controls.

### 17.1 User controls

Players may continue anonymously, opt out of public leaderboards, change or remove a display name, revoke challenges, export application data, and delete a registered account. Export applies spreadsheet-injection protection to user-controlled fields and excludes Auth data, secret codes, tokens, idempotency keys, and encryption material.

### 17.2 Retention and deletion

The retention worker handles expired sessions, challenges, room events, analytics, support records, and anonymous accounts according to documented schedules. Account deletion hides public identity before external Auth cleanup. Backups age out under the provider schedule rather than undergoing unsafe record-level editing.

### 17.3 Processor governance

A production owner must record the selected hosting, authentication, support, error-reporting, email, and analytics processors, including region, DPA, retention, deletion, security review, and accountable owner. The current repository keeps those provider choices launch-blocking instead of embedding unreviewed names in policy text.

## 18. Accessibility, Inclusion, and Localization

The product targets WCAG 2.2 AA. I pair each colour with a stable symbol or pattern, support keyboard play and visible focus, announce accepted feedback through live regions, honour reduced motion, maintain 44 by 44 pixel controls, support zoom and reflow, and provide understandable error states.

### 18.1 Interaction contract

| Need             | Implementation approach                                           | Evidence status                   |
|------------------|-------------------------------------------------------------------|-----------------------------------|
| Keyboard play    | Focusable peg controls, predictable row navigation, submit action | Component tests; E2E spec exists  |
| Screen reader    | Labels, status text, live feedback announcements, route announcer | Component and accessibility specs |
| Colour vision    | Symbols and patterns accompany colour                             | Game board implementation         |
| Reduced motion   | Preference-aware transitions and restrained animation             | CSS and product requirements      |
| Touch and mobile | 44px controls and responsive board                                | Mobile E2E spec exists            |
| Language         | English and Traditional Chinese catalogues                        | Message consistency tests         |

### 18.2 Localization architecture

Locale-prefixed routes and message catalogues separate interface text from component logic. English and Traditional Chinese are supported. Error codes from the API drive localized copy, while the API message remains a safe fallback. Adding a locale requires a complete catalogue, routing update, and localization tests.

### 18.3 Release evidence needed

Automated accessibility checks can catch missing names, contrast failures, and some focus defects. I still need manual keyboard, screen-reader, zoom, mobile reflow, reduced-motion, and colour-vision review in a release browser matrix before launch.

## 19. Analytics, Administration, and Support

Product analytics is first-party, closed-schema, consent-aware, and feature-flagged. The endpoint derives the anonymous flag from the verified principal and accepts documented enums and bounded numbers. It stores no user identifier, IP address, title, guess, secret, invite token, or arbitrary properties object.

### 19.1 Administrative controls

Authorized staff can inspect safe summaries, review games and profiles, moderate names, invalidate or restore leaderboard entries, revoke challenges, terminate rooms, modify typed feature flags, and read audit events. Each mutation requires a reason and records actor, action, target, and timestamp.

### 19.2 Feature flags

Environment flags act as emergency force-off controls. Database flags support controlled runtime changes. A client flag cannot grant capability, and a database flag cannot override an environment force-off. This order lets an operator disable daily, leaderboards, friend challenges, multiplayer, achievements, registration, or analytics during an incident.

### 19.3 Support boundary

Support requests use a bounded API path and a configured support system. Support staff do not receive database or secret-manager access. Engineers should use pseudonymous identifiers and approved operational views; raw production database access remains break-glass, time-bounded, MFA-protected, and audited.

## 20. Command-Line Application and SBA Component

The CLI preserves the educational origin of the project. It imports the same canonical engine, supports official presets and Custom mode, allows computer or human Code Maker play, normalizes input, prints complete history, stores UTF-8 scores in data/high\_scores.csv, sorts and exports scores, and handles quit, EOF, and interruption safely.

### 20.1 Educational value

The CLI makes algorithms and data structures visible without hiding them behind a UI framework. Lists and tuples represent codes and immutable attempts; dictionaries handle mappings; dataclasses and enums model configuration and state; CSV demonstrates file persistence. The duplicate-safe feedback algorithm supplies a worked  $O(n)$  example for an SBA discussion.

### 20.2 Sample interaction

```
Cipherboard CLI Select difficulty: Normal Enter guess: R G B Y Feedback: 1 exact, 2 misplaced  
Attempts remaining: 9 Enter guess: R Y G B Feedback: 4 exact, 0 misplaced You solved the code. Score  
saved.
```

This compact example illustrates the interaction shape. Exact wording can vary with the real CLI prompt and selected configuration.

# 21. Configuration and Environment Management

I separate browser-public settings, API runtime settings, infrastructure credentials, migration-owner credentials, and scheduled-retention credentials. Variables prefixed NEXT\_PUBLIC\_ enter browser JavaScript. They may contain public origins, legal copy, product name, support address, and the Supabase publishable key. Database URLs, service-role credentials, Redis credentials, encryption keys, HMAC keys, and reporting tokens stay server-side.

## 21.1 Production validation

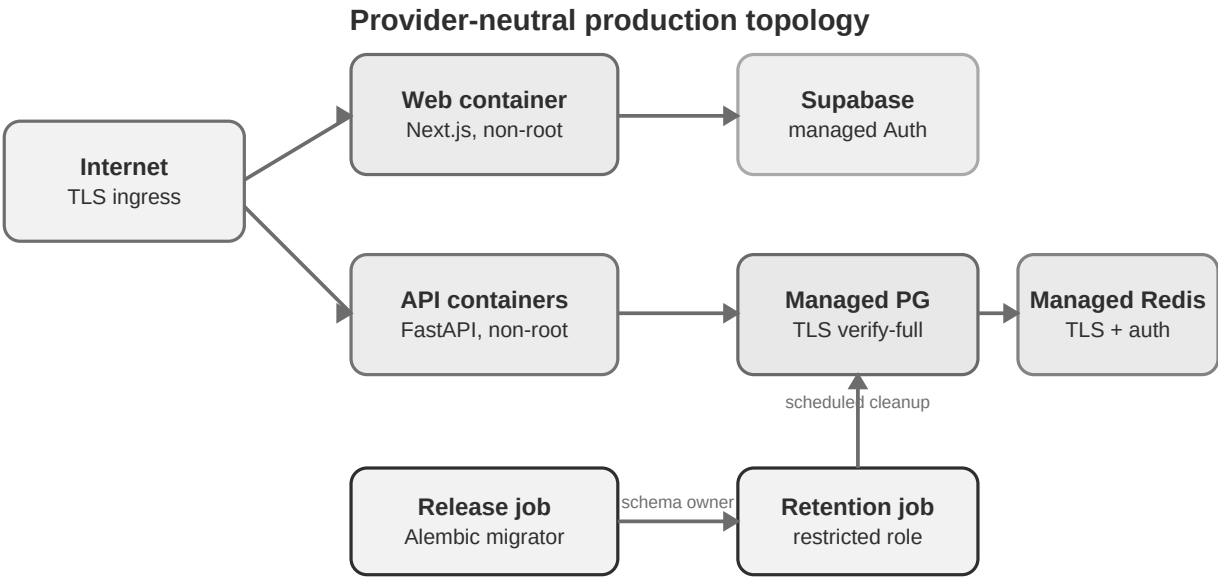
Production and staging startup validate exact HTTPS origins, TLS Redis, non-local database hosts, PostgreSQL verify-full configuration and CA source, key lengths, active versions, release identifier, service credentials, support and legal fields, and safe feature defaults. The API identifies missing variable names without printing values. Operators should correct configuration rather than insert dummy values.

## 21.2 Credential separation

The API receives a least-privilege runtime database URL. Alembic receives a separate migrator URL only in the release job. The retention process receives a restricted role and its own environment file. This separation limits the effect of an API compromise and prevents normal containers from owning the schema.

## 22. Infrastructure and Deployment

The repository supplies multi-stage, non-root API and web containers plus local and production Compose definitions. A production deployment uses managed PostgreSQL and Redis, TLS, protected secrets, exact CORS origins, immutable image identifiers, and a migration step before traffic shifts.



**Figure 8.** The production topology separates public workloads, managed dependencies, schema migration, and scheduled retention credentials.

### 22.1 Release sequence

The release process builds immutable images, validates configuration, backs up and verifies recovery posture, applies Alembic migrations with the migrator identity, starts the API and web containers, checks live and ready endpoints, and then shifts traffic. Post-release checks include guest play, response secret omission, leaderboard integrity, and a two-client duel.

### 22.2 Rollback

Application rollback returns traffic to a recorded image. Database rollback requires a decision based on migration compatibility and data already written by the new version. The runbook favours forward repair when a destructive downgrade would threaten user data. Public web configuration and feature flags can reduce exposure while operators investigate.

## 23. Observability and Operations

Structured logs use request and correlation identifiers, safe route labels, status, duration, release, and bounded operational fields. The API avoids request bodies and recursively redacts sensitive values. Prometheus metrics cover HTTP requests, errors and latency, attempt latency, active games, active rooms, dependency readiness, rate limits, auth failures, and real-time behavior.

### 23.1 Health model

The live endpoint confirms that the process can respond. The ready endpoint checks dependencies and distinguishes failure from degradation. Database failure blocks state-changing gameplay. Redis failure degrades readiness and blocks ranked or race-sensitive operations that cannot preserve coordination integrity.

### 23.2 Alerts and dashboards

The operations documents define example PromQL for request p95, attempt p95, server-error ratio, reconnect rate, current sockets, and public leaderboard p95. Operators still need to tune thresholds against production traffic and avoid high-cardinality labels such as user IDs, room codes, invite capabilities, or request paths containing identifiers.

## 24. Reliability, Recovery, and Incident Response

Cipherboard fails according to data integrity. A database outage rejects mutations. Redis loss leaves committed PostgreSQL outcomes valid but disables operations that depend on distributed coordination. Unverifiable authentication rejects protected access. Analytics delivery may fail without blocking a game. Missing encryption keys return a safe operational error and never trigger replacement-key generation.

### 24.1 Backup and restore

The backup policy calls for managed snapshots or point-in-time recovery, encryption, access controls, and documented retention. A restore drill uses an isolated destination, verifies schema and application integrity, and records recovery evidence. Operators should run the drill each quarter and before relying on a new provider configuration.

### 24.2 Incident response

The incident runbook classifies severity, defines the first 15 minutes, and provides playbooks for secret or key disclosure, JWT compromise, authorization bypass, leaderboard abuse, PostgreSQL failure, Redis or real-time failure, bad deployments, migrations, and privacy exposure. Closure requires recovery verification, evidence preservation, ownership, and follow-up actions.

## 25. Testing and Quality Assurance

I use tests at the domain, API, CLI, component, browser, accessibility, contract, migration, load, and container levels. The local verification for this report covered unit and integration-style tests that do not require an external PostgreSQL service, plus static checks. I record the exact results below.

### Automated verification results

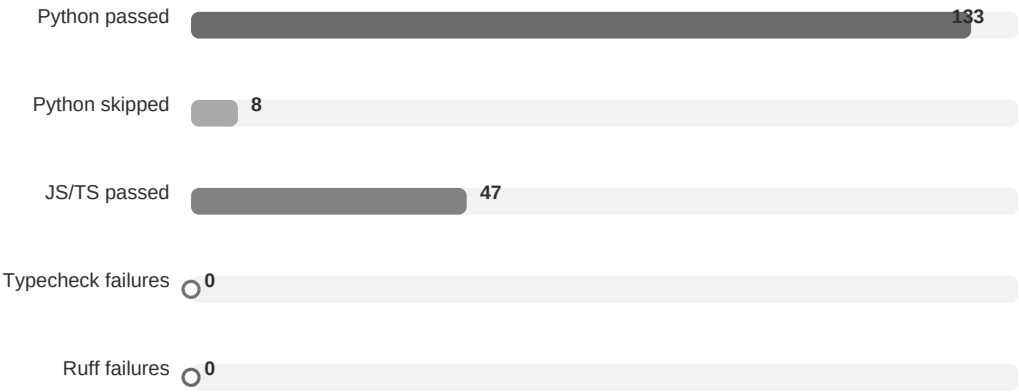


Figure 9. Observed results on 23 August 2026. Skips require their documented integration conditions.

### Measured coverage evidence

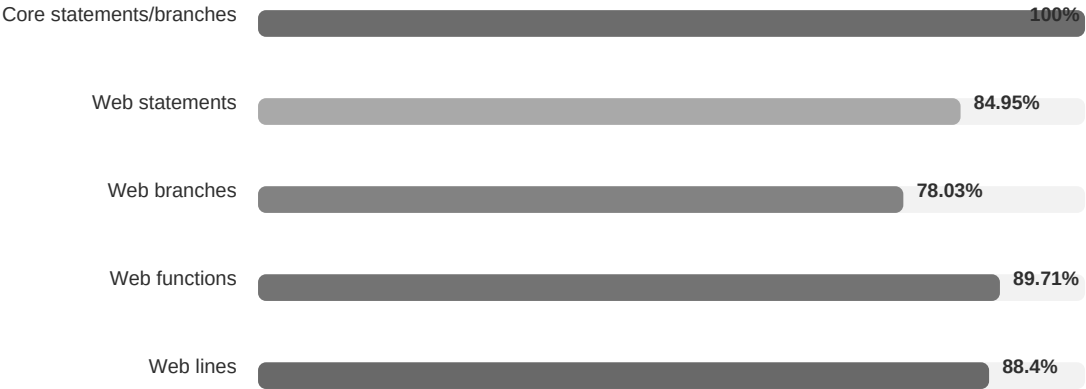


Figure 10. Coverage measured by the executed core pytest-cov and web Vitest suites.

Table 8. Verification evidence

| Check                     | Observed result       | Interpretation                                                           |
|---------------------------|-----------------------|--------------------------------------------------------------------------|
| Python suite              | 133 passed, 8 skipped | Core, API, CLI, security, lifecycle behavior passed in local conditions. |
| Canonical engine coverage | 100%                  | 279 statements and 62 branches measured with no misses.                  |
| JS/TS unit suites         | 47 passed             | Shared config, API client, and eight web test files passed.              |
| Web coverage              | 88.4% lines           | Strong unit coverage; service worker and session paths retain gaps.      |
| TypeScript                | Passed                | Workspace type checking completed with no error.                         |

| Check           | Observed result   | Interpretation                                                   |
|-----------------|-------------------|------------------------------------------------------------------|
| Ruff            | Passed            | Python lint check completed with no error.                       |
| Runtime warning | Node 20.20.0 used | Pinned requirement is Node 24.18.0; repeat release checks there. |

## 25.1 Unexecuted release checks

I did not run the browser E2E, accessibility, build, container, contract-regeneration, load, restore, migration-from-empty, key-rotation, or live duel checks for this report. I therefore do not count them as passed. The repository documents the commands and acceptance criteria for a release environment.

## 26. Performance and Scalability

The product defines targets of LCP p75 below 2.5 seconds, CLS below 0.1, INP below 200 milliseconds, attempt feedback p95 below 250 milliseconds, normal reads and cached leaderboards p95 below 300 milliseconds, and WebSocket propagation p95 below 500 milliseconds. These are targets, not measurements from this report.

### 26.1 Scaling controls

The API uses bounded request bodies, page sizes, timeouts, connection pools, stable indexed leaderboard order, database row locks for critical writes, Redis counters, and horizontal WebSocket fan-out. Stateless HTTP work can scale across API instances, while durable room sequence events support reconnection across instances.

### 26.2 Likely bottlenecks

High-frequency attempt locks, leaderboard scans, connection churn, room replay, and retention queries are the main paths to measure. The existing index plan supports owner history, active expiration, completed sessions, room events, and rank ordering. Production measurements should include database lock wait, pool saturation, Redis latency, socket queue depth, p95 and p99 route latency, and reconnect rate.

## 27. Developer Experience and Maintainability

The monorepo pins Node 24.18.0, pnpm 10.28.2, Python 3.13.14 for production guidance, exact Python dependencies, and a pnpm lockfile. Workspace scripts provide development, type checking, tests, build, E2E, accessibility, load, and API generation commands. uv manages Python environments and Alembic manages schema versions.

### 27.1 Maintainability strengths

- The pure engine isolates game correctness from delivery concerns.
- Explicit schemas and generated client types reduce contract drift.
- Migrations and runbooks turn operational behavior into reviewed source.
- Feature flags and versioned algorithms support controlled change.

### 27.2 Maintenance risks

The FastAPI service layer and room router contain dense transaction and real-time logic. As modes grow, I would extract smaller application services around game creation, attempt submission, room finalization, account deletion, and administrative commands while preserving current transactions. I would also keep OpenAPI generation and schema-drift checks mandatory so manual TypeScript types do not diverge.

## 28. CI/CD and Release Governance

The repository defines formatting, lint, strict typing, unit tests, coverage, builds, browser checks, accessibility, contracts, containers, security scans, and load profiles as release evidence. Production migration uses a protected owner credential and precedes traffic. Deployment environments hold provider credentials and release approvals.

### 28.1 Supply-chain controls

Exact direct dependency versions, lockfiles, dependency review, CodeQL, image scanning, minimal non-root images, immutable image references, and protected CI environments reduce supply-chain risk. An operator should retain SBOM and scan output with the release record.

### 28.2 Go/no-go

The launch checklist requires product configuration, identity, privacy, game integrity, secret management, accessibility, reliability, observability, supply-chain, legal, backup, and load evidence. I would require an accountable reviewer to sign each evidence link and reject a release that substitutes assumptions or local unit results for managed-environment checks.

## 29. System Assessment

Cipherboard aligns well with its functional requirements. The canonical engine, server-authoritative API, database constraints, private multiplayer model, identity lifecycle, and operational documents form a coherent system. The implementation also gives privacy, accessibility, and moderation enough architectural weight to avoid retrofitting them after launch.

*Table 9. Production readiness assessment*

| Area               | Assessment                          | Reason                                                                         |
|--------------------|-------------------------------------|--------------------------------------------------------------------------------|
| Domain correctness | Strong                              | Pure engine, 100% measured coverage, boundary and feedback tests.              |
| API integrity      | Strong                              | Explicit schemas, ownership predicates, locks, idempotency, safe errors.       |
| Data model         | Strong                              | Constraints, versions, indexes, migration authority, retention design.         |
| Security design    | Strong with deployment dependencies | Threat model and layered controls; managed evidence remains required.          |
| Privacy            | Strong design                       | Data inventory, exclusions, export, deletion, retention; legal review pending. |
| Accessibility      | Promising                           | Clear contract and tests; manual release review not run here.                  |
| Operations         | Documented                          | Runbooks exist; drills and production metrics not supplied.                    |
| Performance        | Unverified target                   | Load profiles and targets exist; no live measurements in this report.          |
| Release readiness  | Conditional                         | Local checks passed; pinned-runtime and managed-service gates remain.          |

### 29.1 Key strengths

The system makes authority explicit. The browser cannot approve a guess; Redis cannot rewrite a winner; display metadata cannot grant admin rights; analytics cannot accept arbitrary properties; and a missing key cannot cause secret regeneration. These boundaries reduce ambiguity during both normal operation and incidents.

### 29.2 Main gaps

The report lacks production performance data, completed restore and key drills, final legal processor choices, release-browser evidence, and results from the external integration suite. The local Node mismatch also means the web checks need repetition under the pinned version before release approval.

## 30. Recommendations and Roadmap

Table 10. Prioritized recommendations

| Priority | Action                                                                  | Acceptance evidence                                                          |
|----------|-------------------------------------------------------------------------|------------------------------------------------------------------------------|
| P0       | Repeat full validation on Node 24.18.0 and Python 3.13.14.              | All static, test, build, E2E, and accessibility jobs pass.                   |
| P0       | Run PostgreSQL migration and integration suites from an empty database. | Migration current, downgrade decision recorded, all integration tests pass.  |
| P0       | Complete managed deployment configuration and legal review.             | Launch checklist links, processor record, policy approval, no placeholders.  |
| P0       | Perform restore and key-rotation drills.                                | Timed drill record, integrity checks, owner sign-off, follow-up actions.     |
| P1       | Run load and resilience profiles against staging.                       | p95/p99, saturation, lock-wait, reconnect, and error evidence.               |
| P1       | Complete manual accessibility review.                                   | Keyboard, screen reader, zoom, mobile, reduced motion, colour review.        |
| P1       | Run one two-client real-time smoke across reconnect.                    | Ticket, replay, privacy, tie window, and winner behavior recorded.           |
| P2       | Refactor dense application services behind smaller boundaries.          | No contract change; transaction tests remain green; lower review complexity. |
| P2       | Add production dashboard and alert tuning after traffic baseline.       | Alert tests, bounded labels, operator ownership, low false-positive rate.    |

### 30.1 Suggested delivery order

I would first close correctness and environment gaps, then produce recovery and security evidence, and then tune performance and operations against staging traffic. Product expansion should follow those gates so new modes do not widen an unverified operational surface.

# 31. Conclusion

Cipherboard demonstrates how a familiar logic game can become a disciplined software system. I began with the feedback algorithm and game loop, then separated the domain from API, web, identity, persistence, and real-time delivery. That separation gave me one rule source for the CLI and server while keeping competitive authority away from the browser.

The repository now covers anonymous onboarding, official and custom games, daily play, friend challenges, private duels, profiles, achievements, leaderboards, export, deletion, administration, privacy, accessibility, localization, deployment, observability, and incident response. The tests I ran support the core correctness and unit-level quality claims.

My final assessment is conditional readiness. The architecture and implementation are strong enough to justify staging release work. Public production approval still requires the pinned runtime, external services, browser and accessibility evidence, legal configuration, security and recovery drills, and measured load results. I would keep those requirements as launch gates because they protect player data, fair outcomes, and the credibility of the product.

## Student reflection

The project taught me that system quality comes from explicit boundaries. Algorithms, identity, data, security, user experience, and operations must agree on who owns each decision.

# A. Appendix A - Repository Map

| Path                     | Purpose                                                                |
|--------------------------|------------------------------------------------------------------------|
| apps/web                 | Next.js App Router PWA, components, localization, unit and E2E tests   |
| apps/api                 | FastAPI, SQLAlchemy models, routers, services, Alembic migrations      |
| apps/cli                 | Python SBA command-line application and local CSV storage              |
| packages/mastermind_core | Canonical pure Python rules and scoring                                |
| packages/api_client      | Generated TypeScript API contract                                      |
| packages/shared_config   | Browser-safe shared product configuration                              |
| tests                    | Python core, API, CLI, load, migration, and security tests             |
| infra                    | Container definitions and guarded development/operations scripts       |
| docs                     | Architecture, product, security, privacy, operations, and SBA evidence |
| supabase                 | Local Supabase project configuration and guidance                      |

## A.1 Primary implementation files

**Domain:** feedback.py, engine.py, validation.py, scoring.py, daily.py, presets.py, and models.py. **Backend:** main.py, auth.py, services.py, realtime.py, crypto.py, models.py, config.py, and routers. **Web:** game-board.tsx, session-provider.tsx, feature-page.tsx, app routes, API helpers, preferences, analytics delivery, service worker, and locale catalogues.

# B. Appendix B - API Route Catalogue

| Area       | Method | Route                            |
|------------|--------|----------------------------------|
| Games      | POST   | /v1/games                        |
| Games      | GET    | /v1/games/{gameId}               |
| Games      | POST   | /v1/games/{gameId}/attempts      |
| Games      | POST   | /v1/games/{gameId}/abandon       |
| Daily      | GET    | /v1/daily                        |
| Daily      | POST   | /v1/daily/start                  |
| Daily      | GET    | /v1/daily/leaderboard            |
| Challenges | POST   | /v1/challenges                   |
| Challenges | GET    | /v1/challenges/mine              |
| Challenges | GET    | /v1/challenges/{shareCode}       |
| Challenges | POST   | /v1/challenges/{shareCode}/start |
| Challenges | GET    | /v1/challenges/{id}/results      |
| Challenges | DELETE | /v1/challenges/{id}              |
| Rooms      | POST   | /v1/rooms                        |
| Rooms      | POST   | /v1/rooms/{code}/join            |
| Rooms      | GET    | /v1/rooms/{id}                   |
| Rooms      | POST   | /v1/rooms/{id}/ready             |
| Rooms      | POST   | /v1/rooms/{id}/ws-ticket         |
| Rooms      | WS     | /v1/rooms/{id}/events            |
| Player     | GET    | /v1/me/profile                   |
| Player     | PATCH  | /v1/me/profile                   |
| Player     | GET    | /v1/me/stats                     |
| Player     | GET    | /v1/me/games                     |
| Player     | GET    | /v1/me/export                    |
| Player     | DELETE | /v1/me                           |
| Public     | GET    | /v1/leaderboards                 |
| Analytics  | POST   | /v1/analytics/events             |
| Support    | POST   | /v1/support                      |
| Operations | GET    | /health/live                     |
| Operations | GET    | /health/ready                    |
| Operations | GET    | /metrics                         |
| Admin      | GET    | /v1/admin/summary                |
| Admin      | GET    | /v1/admin/games                  |
| Admin      | GET    | /v1/admin/profiles               |
| Admin      | GET    | /v1/admin/rooms                  |
| Admin      | GET    | /v1/admin/challenges             |
| Admin      | GET    | /v1/admin/leaderboard-review     |

| Area  | Method    | Route                                 |
|-------|-----------|---------------------------------------|
| Admin | GET/PATCH | /v1/admin/flags[/key]                 |
| Admin | POST      | /v1/admin/leaderboard/{id}/invalidate |
| Admin | POST      | /v1/admin/leaderboard/{id}/restore    |
| Admin | POST      | /v1/admin/profiles/{id}/moderate      |
| Admin | POST      | /v1/admin/challenges/{id}/revoke      |
| Admin | POST      | /v1/admin/rooms/{id}/terminate        |
| Admin | GET       | /v1/admin/audit                       |

FastAPI publishes OpenAPI for JSON routes. The TypeScript client package derives from that contract. The WebSocket route uses its own versioned command and event schema.

## C. Appendix C - Data Entity Catalogue

| ORM class              | Table                     | Data class                      |
|------------------------|---------------------------|---------------------------------|
| Profile                | profiles                  | Authoritative PostgreSQL record |
| DailyChallenge         | daily_challenges          | Authoritative PostgreSQL record |
| FriendChallenge        | friend_challenges         | Authoritative PostgreSQL record |
| MultiplayerRoom        | multiplayer_rooms         | Authoritative PostgreSQL record |
| GameSession            | game_sessions             | Authoritative PostgreSQL record |
| GameAttempt            | game_attempts             | Authoritative PostgreSQL record |
| MultiplayerMember      | multiplayer_members       | Authoritative PostgreSQL record |
| MultiplayerEvent       | multiplayer_events        | Authoritative PostgreSQL record |
| LeaderboardEntry       | leaderboard_entries       | Authoritative PostgreSQL record |
| Achievement            | achievements              | Authoritative PostgreSQL record |
| UserAchievement        | user_achievements         | Authoritative PostgreSQL record |
| FeatureFlag            | feature_flags             | Authoritative PostgreSQL record |
| ModerationAction       | moderation_actions        | Authoritative PostgreSQL record |
| AuditEvent             | audit_events              | Authoritative PostgreSQL record |
| AdminGrant             | admin_grants              | Authoritative PostgreSQL record |
| SupportRequest         | support_requests          | Authoritative PostgreSQL record |
| AccountDeletionRequest | account_deletion_requests | Authoritative PostgreSQL record |
| ProductEvent           | product_events            | Authoritative PostgreSQL record |

## D. Appendix D - Environment Variable Groups

| Group         | Examples                                                                                                         | Boundary                                       |
|---------------|------------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| Public web    | NEXT_PUBLIC_API_ORIGIN, PRODUCT_ORIGIN, SUPABASE_URL, PUBLISHABLE_KEY, PRODUCT_NAME, SUPPORT_EMAIL, legal fields | Embedded in browser bundle; no secrets         |
| API runtime   | MASTERMIND_ENVIRONMENT, DATABASE_URL, REDIS_URL, ALLOWED_ORIGINS, SUPABASE_URL, rate and socket limits           | Server container only                          |
| Cryptographic | SECRET_ENCRYPTION_KEYS, DAILY_HMAC_KEYS, PUBLIC_IDENTIFIER_HMAC_KEY                                              | Managed secret store; independent key material |
| Release       | DATABASE_MIGRATOR_URL, MASTERMIND_RELEASE, image references                                                      | Protected release job                          |
| Retention     | DATABASE_RETENTION_URL                                                                                           | Scheduled job with restricted role             |
| Local Compose | POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD                                                                    | Local development inputs                       |

Production startup validates required fields and rejects placeholder or unsafe values. The full variable-by-variable reference remains in docs/operations/ENVIRONMENT\_VARIABLES.md.

## E. Appendix E - Error and Safety Contract

JSON errors use the fields **code**, **message**, and **requestId**. The code remains stable for localization and program logic. The message stays safe for display. The response excludes exception traces, SQL, ciphertext, secrets, credentials, environment values, and internal object state.

```
{ "code": "INVALID_GUESS_LENGTH", "message": "Your guess must contain exactly four pegs.",  
  "requestId": "01J..." }
```

### E.1 Representative stable codes

AUTHENTICATION\_REQUIRED, INVALID\_ACCESS\_TOKEN, ADMIN\_REQUIRED, MFA\_REQUIRED,  
RECENT\_AUTH\_REQUIRED, GAME\_NOT\_ACTIVE, INVALID\_GUESS\_LENGTH, REQUEST\_TOO\_LARGE,  
REQUEST\_TIMEOUT, DEDICATED\_ENDPOINT\_REQUIRED, SECRET\_NOT\_ACCEPTED,  
INVALID\_CONTENT\_LENGTH, and feature-specific unavailable or not-found codes.

## F. Appendix F - Verification Record

**Environment date:** 23 August 2026. **Platform:** macOS. **Python test runtime:** 3.13.14. **Node runtime:** 20.20.0 with repository engine warning. **pnpm:** 10.28.2.

| Check                | Exit | Result detail                                                                    |
|----------------------|------|----------------------------------------------------------------------------------|
| uv run pytest        | 0    | 141 collected; 133 passed; 8 skipped; 13.66 seconds reported by pytest.          |
| Core pytest-cov gate | 0    | 42 passed; 279 statements; 62 branches; 100% total coverage.                     |
| uv run ruff check .  | 0    | All checks passed.                                                               |
| pnpm typecheck       | 0    | API client, shared config, and web type checks passed.                           |
| pnpm test            | 0    | 1 API client file, 1 shared config file, and 8 web files passed; 47 tests total. |

I did not count skipped or unexecuted checks as passing. The release owner should attach CI URLs, browser artifacts, accessibility output, migration evidence, container scan results, load reports, restore records, key-rotation records, and live smoke evidence to a production release.

## G. Appendix G - References

| Ref | Source                                               | Use in report                                                                            |
|-----|------------------------------------------------------|------------------------------------------------------------------------------------------|
| R1  | README.md                                            | Product overview, architecture, setup, validation, deployment, and troubleshooting.      |
| R2  | PRODUCT.md                                           | Product purpose, users, design principles, and accessibility intent.                     |
| R3  | docs/product/PRODUCT_REQUIREMENTS.md                 | Rules, modes, profiles, administration, and quality targets.                             |
| R4  | docs/architecture/SYSTEM_OVERVIEW.md                 | Runtime boundaries, request flow, versioning, and failure model.                         |
| R5  | docs/architecture/API_DESIGN.md                      | Routes, authentication, idempotency, pagination, and secret omission.                    |
| R6  | docs/architecture/DATA_MODEL.md                      | Entities, invariants, indexes, deletion, and migration ownership.                        |
| R7  | docs/architecture/REALTIME.md                        | Tickets, commands, events, replay, presence, and winner flow.                            |
| R8  | docs/architecture/SCORING_AND_RANKING.md             | score_v1, eligibility, ties, periods, and review.                                        |
| R9  | docs/security/THREAT_MODEL.md                        | Assets, trust boundaries, abuse cases, controls, and assumptions.                        |
| R10 | docs/security/SECURITY_CONTROLS.md                   | Implemented security controls and launch checks.                                         |
| R11 | docs/privacy/DATA_INVENTORY.md                       | Collected data, exclusions, processors, and user controls.                               |
| R12 | docs/privacy/RETENTION.md                            | Retention, anonymous cleanup, deletion, and backup treatment.                            |
| R13 | docs/product/ACCESSIBILITY.md and<br>LOCALIZATION.md | Interaction, visual, manual review, and locale rules.                                    |
| R14 | docs/operations/*.md                                 | Deployment, verification, testing, observability, recovery, keys, incidents, and launch. |
| R15 | docs/sba/SBA_IMPLEMENTATION.md                       | Educational decomposition, algorithms, CLI, CSV, and tests.                              |
| R16 | apps/, packages/, tests/, infra/                     | Implementation and automated verification source.                                        |

# H. Appendix H - Glossary

| Term               | Meaning                                                                               |
|--------------------|---------------------------------------------------------------------------------------|
| AAL2               | Authentication assurance level requiring a second factor for administrative access.   |
| AES-256-GCM        | Authenticated encryption used for persisted game secrets.                             |
| Capability token   | High-entropy secret URL or room value whose possession grants a narrow action.        |
| Canonical engine   | The sole framework-independent implementation of Mastermind rules.                    |
| CSP                | Content Security Policy response header that restricts browser resource behavior.     |
| Idempotency        | Property that lets a safe retry return the original effect instead of duplicating it. |
| JWKS               | Provider key set used to verify JWT signatures.                                       |
| PITR               | Point-in-time database recovery.                                                      |
| PWA                | Installable progressive web application with a service-worker-backed shell.           |
| Ranked eligibility | Server state that determines whether an outcome may enter competition.                |
| Source of truth    | System whose committed record decides authoritative state.                            |
| WSS                | WebSocket over TLS.                                                                   |



# PART II

## Development Evidence Supplement

Added after comparison with a successful ICT project report

Problem definition | Analysis | Design | Testing | Debugging | User guide | Timeline

Prepared in the first-person student perspective

Cipherboard repository baseline: 3efefdb

Evidence refreshed: 23 August 2026

# Supplement Contents

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>Supplement Contents</b>                                  | <b>55</b> |
| <b>Reference Gap Analysis</b>                               | <b>57</b> |
| <b>S1. Problem Definition</b>                               | <b>58</b> |
| S1.1 Background . . . . .                                   | 58        |
| S1.2 Problem statement . . . . .                            | 58        |
| S1.3 Objectives . . . . .                                   | 58        |
| S1.4 Minimum expectations . . . . .                         | 58        |
| S1.5 System development cycle . . . . .                     | 59        |
| <b>S2. System Analysis and Technology Selection</b>         | <b>60</b> |
| S2.1 Programming language analysis . . . . .                | 60        |
| S2.2 User-interface analysis . . . . .                      | 60        |
| S2.3 Persistence and coordination analysis . . . . .        | 60        |
| S2.4 Framework, tooling, and environment decision . . . . . | 60        |
| S2.5 Deployment alternatives . . . . .                      | 60        |
| <b>S3. Problem Analysis</b>                                 | <b>61</b> |
| S3.1 Users and operational actors . . . . .                 | 61        |
| S3.2 Input-process-output cycle . . . . .                   | 61        |
| S3.3 Processes . . . . .                                    | 61        |
| S3.4 Outputs . . . . .                                      | 62        |
| S3.5 Constraints and edge cases . . . . .                   | 62        |
| S3.6 Assumptions . . . . .                                  | 62        |
| <b>S4. Program Design</b>                                   | <b>63</b> |
| S4.1 Modularization . . . . .                               | 63        |
| S4.2 Overall gameplay flow . . . . .                        | 63        |
| S4.3 Navigation and interface structure . . . . .           | 64        |
| S4.4 Attempt-submission pseudocode . . . . .                | 64        |
| S4.5 Data dictionary . . . . .                              | 64        |
| S4.6 Validation design . . . . .                            | 64        |
| <b>S5. Design Implementation</b>                            | <b>65</b> |
| S5.1 Initialization and configuration . . . . .             | 65        |
| S5.2 Data structures . . . . .                              | 65        |
| S5.3 Core models in source . . . . .                        | 65        |
| S5.4 Classes and modules . . . . .                          | 66        |
| S5.5 Function catalogue . . . . .                           | 66        |
| S5.6 Libraries and dependencies . . . . .                   | 66        |
| S5.7 Test implementation example . . . . .                  | 67        |
| <b>S6. Testing and Evaluation Evidence</b>                  | <b>68</b> |

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| S6.1 Testing plan . . . . .                                      | 68        |
| S6.2 Observed command evidence . . . . .                         | 68        |
| S6.3 Phase 1 cases: initialization and boundaries . . . . .      | 68        |
| S6.4 Phase 2 cases: algorithms and integrated behavior . . . . . | 69        |
| S6.5 Phase 3 cases: outputs and acceptance . . . . .             | 69        |
| S6.6 Testing evaluation . . . . .                                | 69        |
| <b>S7. Debugging and Implemented Improvements</b>                | <b>70</b> |
| S7.1 Debugging method . . . . .                                  | 70        |
| S7.2 Implemented improvements . . . . .                          | 70        |
| S7.3 Further improvement list . . . . .                          | 70        |
| <b>S8. User Manual</b>                                           | <b>71</b> |
| S8.1 System requirements . . . . .                               | 71        |
| S8.2 Starting the system . . . . .                               | 71        |
| S8.3 Playing a solo game . . . . .                               | 71        |
| S8.4 Daily, challenge, and duel workflows . . . . .              | 71        |
| S8.5 Profiles, accessibility, and privacy controls . . . . .     | 71        |
| S8.6 Troubleshooting . . . . .                                   | 71        |
| S8.7 Frequently asked questions . . . . .                        | 72        |
| <b>S9. Program Evaluation and Student Reflection</b>             | <b>73</b> |
| S9.1 Evaluation against objectives . . . . .                     | 73        |
| S9.2 Benefits and limitations . . . . .                          | 73        |
| S9.3 Student reflection . . . . .                                | 73        |
| S9.4 Future development . . . . .                                | 73        |
| <b>S10. Project Management and Timeline</b>                      | <b>74</b> |
| S10.1 Milestone record . . . . .                                 | 74        |
| S10.2 Risk tracking . . . . .                                    | 74        |
| <b>S11. References, AI Use, and Evidence Integrity</b>           | <b>75</b> |
| S11.1 Reference report use . . . . .                             | 75        |
| S11.2 Cipherboard sources . . . . .                              | 75        |
| S11.3 AI-tool disclosure . . . . .                               | 75        |
| S11.4 Evidence-integrity statement . . . . .                     | 75        |
| <b>End of Report</b>                                             | <b>76</b> |

# Reference Gap Analysis

I treated the attached successful report as a structural benchmark. I did not treat its descriptions, code, or conclusions as instructions for Cipherboard. The comparison showed that my original report explained the finished architecture in depth but recorded less of the student development process.

| Missing or thin area                        | Evidence added in this supplement                                               |
|---------------------------------------------|---------------------------------------------------------------------------------|
| Problem definition and minimum expectations | S1 background, problem statement, objectives, expectations, development cycle   |
| Technology alternatives                     | S2 language, interface, persistence, tooling, and deployment comparisons        |
| Input-output and constraint analysis        | S3 stakeholders, IPO cycle, inputs, processes, outputs, assumptions, edge cases |
| Detailed program design                     | S4 module hierarchy, navigation, flowcharts, pseudocode, data dictionary        |
| Implementation catalogue                    | S5 structures, variables, classes, functions, libraries, configuration          |
| Case-by-case testing                        | S6 phased plan, expected-versus-actual tables, pending acceptance work          |
| Debugging record                            | S7 six fixes traced to repository commits and retest evidence                   |
| User manual                                 | S8 installation, flows, controls, privacy actions, troubleshooting, FAQ         |
| Objective evaluation and reflection         | S9 objective matrix, benefits, limits, learning outcomes                        |
| Project timeline and AI-use record          | S10 reconstructed Gantt; S11 sources and tool disclosure                        |

The original system report remains Part I. This supplement supplies the missing lifecycle evidence without duplicating the architecture, security, privacy, and operations chapters already present.

# S1. Problem Definition

## S1.1 Background

Mastermind has a small rule set, but a complete online product introduces identity, persistence, competition, social play, accessibility, security, and recovery. A browser-only implementation can display a board, yet it cannot protect a secret or prove that a leaderboard score came from valid guesses. I selected the problem because it let me begin with a clear algorithm and expand it into a full system.

## S1.2 Problem statement

I needed to create one accessible Mastermind system that supports immediate guest play, permanent accounts, official and custom modes, daily competition, friend challenges, private live duels, a local CLI, and production operations. The system had to keep active secrets away from the browser and make PostgreSQL the authority for accepted attempts and outcomes.

## S1.3 Objectives

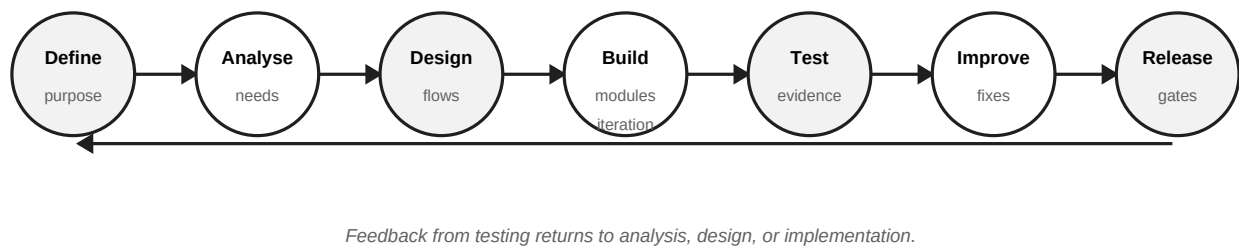
| ID  | Objective                                                                         | Success evidence                                            |
|-----|-----------------------------------------------------------------------------------|-------------------------------------------------------------|
| O1  | Calculate exact and misplaced feedback without double counting.                   | Core examples, exhaustive duplicate tests, invariants       |
| O2  | Let a first-time player begin through authenticated guest identity.               | Session provider, JWT path, product-flow tests              |
| O3  | Support official, practice, custom, daily, friend, pass-and-play, and duel modes. | Routes, UI pages, API tests                                 |
| O4  | Preserve competitive fairness through server authority.                           | Secret omission, row locks, score_v1, unique outcomes       |
| O5  | Restore games and live rooms after refresh or reconnect.                          | Persistent attempts, sequence replay, room tests            |
| O6  | Provide profiles, statistics, achievements, and moderated leaderboards.           | Profile/admin tests and data model                          |
| O7  | Provide export and account deletion controls.                                     | Lifecycle tests and retention worker                        |
| O8  | Meet the defined accessibility and localization contract.                         | Keyboard/symbol design, two catalogues, test specifications |
| O9  | Protect secrets, identity, and administrative authority.                          | Encryption, JWT, authorization, audit tests                 |
| O10 | Support controlled deployment, monitoring, backup, and incident response.         | Containers, migrations, metrics, runbooks                   |

## S1.4 Minimum expectations

- A valid game starts, accepts legal guesses, rejects invalid guesses without consuming an attempt, and ends in a reproducible state.
- The client does not receive an active secret or choose accepted feedback, score, rank, or duel winner.
- A player can understand the board through text and symbols without relying on colour.
- The CLI can play and store local results without web infrastructure.
- The service fails safely when authentication, persistence, coordination, or key material cannot support a trusted mutation.

S1.5 System development cycle

Cipherboard system development cycle



Supplement Figure 1. The project cycle links definition, analysis, design, implementation, verification, improvement, and release gates.

## S2. System Analysis and Technology Selection

### S2.1 Programming language analysis

No single language served every layer well. I chose Python for the canonical rules, API, and SBA CLI because it expresses algorithms clearly and has mature FastAPI and SQLAlchemy support. I chose TypeScript for the Next.js client because compile-time checking helps keep UI state and API contracts aligned.

| Option          | Strengths for this project                                  | Limitations                                              | Decision                         |
|-----------------|-------------------------------------------------------------|----------------------------------------------------------|----------------------------------|
| Python          | Readable algorithms, strong testing, FastAPI, SBA relevance | Less suitable for a typed React browser client           | Use for core, API, CLI           |
| TypeScript/Node | Strong web tooling, shared browser types, React ecosystem   | Would duplicate the school-facing Python engine          | Use for web and generated client |
| Java            | Strong types and mature server tooling                      | More ceremony for the learning-focused engine and CLI    | Rejected for this scope          |
| C#              | Good desktop and server ecosystem                           | Weaker fit for the chosen PWA and Python SBA requirement | Rejected for this scope          |

### S2.2 User-interface analysis

| Interface          | Benefit                                                                   | Cost                                                 | Decision                     |
|--------------------|---------------------------------------------------------------------------|------------------------------------------------------|------------------------------|
| CLI                | Low system demand; exposes algorithms; supports SBA evidence              | Limited discoverability and no network competition   | Retain as educational client |
| Desktop GUI        | Rich local controls and offline potential                                 | Installation and platform packaging burden           | Not selected                 |
| Responsive web PWA | Works on mobile and desktop; installable; supports accounts and live play | Requires browser, API, and network-resilience design | Primary interface            |

### S2.3 Persistence and coordination analysis

| Technology | Appropriate use                         | Reason for final boundary                                       |
|------------|-----------------------------------------|-----------------------------------------------------------------|
| CSV        | Local CLI high scores                   | Portable and easy to inspect; unsuitable for shared concurrency |
| SQLite     | Tests and isolated development          | Simple local database; limited production concurrency semantics |
| PostgreSQL | Authoritative product state             | Transactions, constraints, row locking, indexes, durable replay |
| Redis      | Rate limits, tickets, presence, fan-out | Fast coordination; excluded from authoritative outcomes         |

### S2.4 Framework, tooling, and environment decision

FastAPI and Pydantic provide a versioned OpenAPI contract. SQLAlchemy and Alembic separate runtime access from schema ownership. Next.js provides route composition, server rendering, localization support, and a PWA shell. Vitest, Playwright, pytest, Ruff, mypy, TypeScript, k6, Docker, and GitHub Actions cover different verification layers. The repository stays editor-neutral, so a developer may use VS Code, PyCharm, or another IDE without changing the build contract.

### S2.5 Deployment alternatives

| Alternative                                  | Assessment                                                                                                       |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Single local process                         | Useful for CLI and development; cannot provide shared identity or durable competition.                           |
| Serverless-only data path                    | Reduces host management but complicates long-lived sockets, migrations, and provider-neutral recovery.           |
| OCI containers with managed PostgreSQL/Redis | Selected documented path. It preserves portability while delegating backup and availability to managed services. |

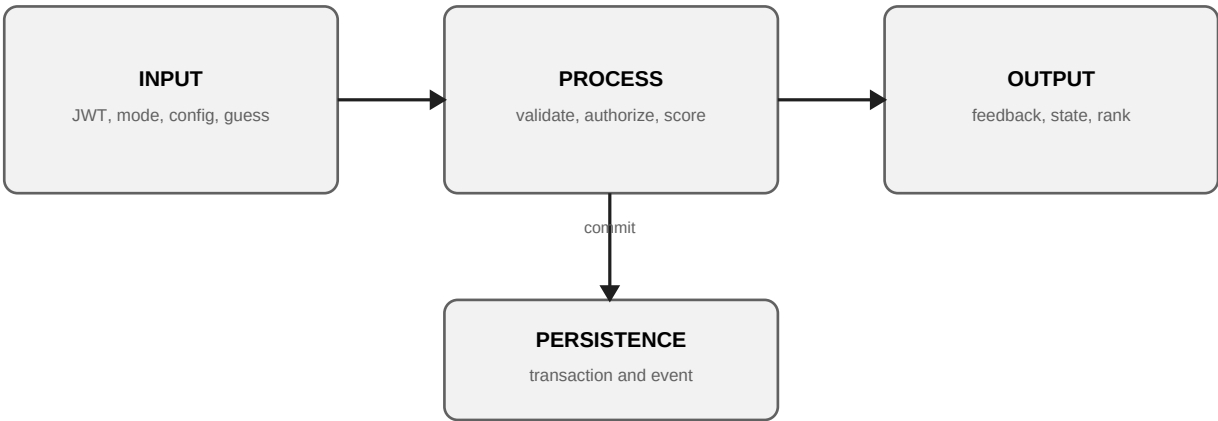
## S3. Problem Analysis

### S3.1 Users and operational actors

The direct actors are guest players, registered players, challenge creators, room members, administrators, support staff, release operators, and scheduled maintenance jobs. Each actor receives a separate authority boundary. A public capability can reveal a challenge description, while creator management still requires identity and ownership.

### S3.2 Input-process-output cycle

Input-process-output cycle for one submitted guess



Supplement Figure 2. A submitted guess crosses authentication, validation, domain logic, and persistence before the API returns safe output.

| Input class        | Examples                                                | Validation                                          |
|--------------------|---------------------------------------------------------|-----------------------------------------------------|
| Identity           | Bearer JWT, WebSocket ticket, session ID                | Signature, issuer, audience, expiry, one-use scope  |
| Game configuration | Mode, preset, palette, length, attempts, duplicates     | Enum and numeric bounds; official preset equality   |
| Gameplay           | Guess sequence, idempotency key                         | Length, known colours, active state, bounded key    |
| Social             | Title, display-name option, room code, share capability | Length, Unicode safety, digest lookup, membership   |
| Account            | Profile changes, consent, DELETE confirmation           | Ownership, recent auth, exact confirmation          |
| Administration     | Target, action, reason, flag value                      | Grant, AAL2, recent auth, typed value, audit reason |

### S3.3 Processes

- Authenticate the caller and construct one principal.
- Authorize the requested game, challenge, room, profile, or administrative target in the query.
- Normalize and validate input before a state transition.
- Lock the authoritative row for race-sensitive changes, apply the canonical engine, write immutable attempts, and commit.
- Publish safe real-time events after commit and return an explicit response model.

## S3.4 Outputs

| Output                  | Audience                 | Safety rule                                                                 |
|-------------------------|--------------------------|-----------------------------------------------------------------------------|
| Game state and feedback | Owner                    | No active secret; accepted server feedback only                             |
| Leaderboard row         | Public opt-in viewers    | Moderation-aware fields; no user ID or guess history                        |
| Room event              | Authorized members       | Presence and progress only; private guesses omitted                         |
| Export ZIP              | Registered account owner | No Auth data, secret, invite token, idempotency key, or encryption material |
| Metrics/logs            | Operators                | Bounded labels, request IDs, recursive redaction                            |

## S3.5 Constraints and edge cases

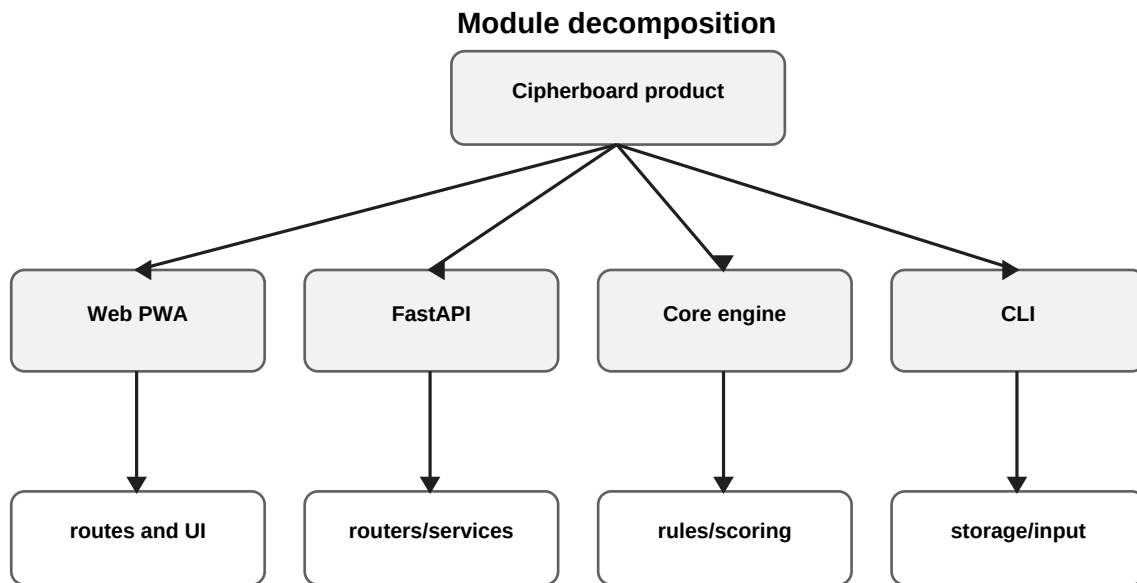
| Constraint                 | Required handling                                                                    |
|----------------------------|--------------------------------------------------------------------------------------|
| Duplicate colours          | Exact matches first; unmatched Counter intersection; no peg counted twice            |
| Concurrent retries         | Row lock, request fingerprint, unique constraints, stored original outcome           |
| Midnight during daily run  | Keep the session on its original UTC challenge date                                  |
| Redis outage               | Keep PostgreSQL outcomes valid; fail closed for ranked and race-sensitive operations |
| Refresh or reconnect       | Restore from API state and durable room sequence                                     |
| Deleted or banned identity | Hide public data and deny protected operations                                       |
| Malformed Unicode          | Normalize NFKC and reject control or bidirectional formatting characters             |
| Missing key version        | Return safe retryable service error; do not generate a replacement                   |

## S3.6 Assumptions

Production assumes modern TLS ingress, private managed PostgreSQL and Redis, protected secrets, MFA for operators, working backups, and reviewed legal configuration. If a deployment cannot meet an assumption, the launch checklist requires a tested compensating control or blocks release.

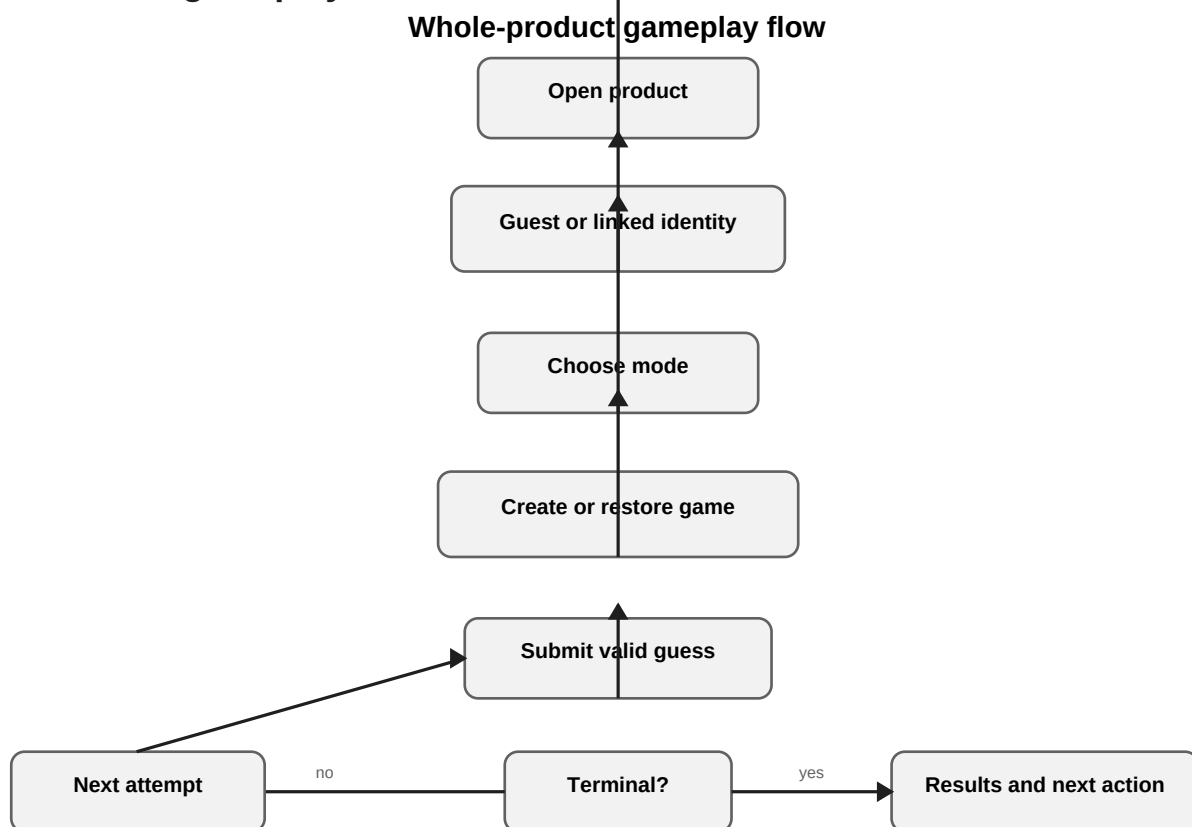
## S4. Program Design

### S4.1 Modularization



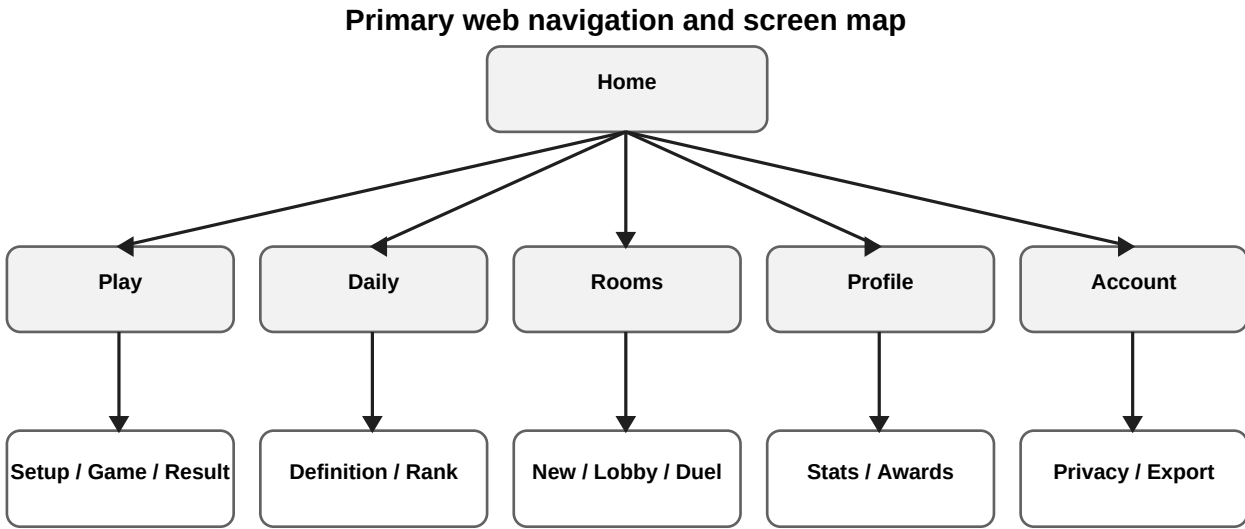
*Supplement Figure 3. The product separates presentation, delivery, rules, and local educational interaction.*

### S4.2 Overall gameplay flow



*Supplement Figure 4. Every mode converges on authoritative creation or restoration, valid attempts, and a terminal result.*

### S4.3 Navigation and interface structure



Supplement Figure 5. Primary route groups and their child screens.

### S4.4 Attempt-submission pseudocode

```

    REQUIRE authenticated principal LOAD owned game FOR UPDATE IF idempotency key already exists: REQUIRE
    payload fingerprint matches RETURN stored response REQUIRE game is active and before expiry DECRYPT secret
    using recorded key version new_state = core.submit_guess(state, secret, guess) INSERT immutable attempt
    UPDATE game state, score, eligibility UPSERT awards and leaderboard when eligible COMMIT transaction
    PUBLISH safe event RETURN explicit response

```

### S4.5 Data dictionary

| Name               | Type                         | Meaning                                             | Invariant                                 |
|--------------------|------------------------------|-----------------------------------------------------|-------------------------------------------|
| config             | GameConfig / JSON            | Palette, length, limit, duplicate and rank settings | Validated and versioned                   |
| secret             | tuple[str] / encrypted bytes | Code Maker sequence                                 | Plaintext stays in trusted process memory |
| guess              | tuple[str] / JSON array      | Normalized player attempt                           | Length equals code length                 |
| attempts           | immutable tuple / table rows | Ordered valid attempts                              | One number and idempotency key per game   |
| status             | GameStatus enum / string     | created, active, won, lost, abandoned, expired      | Only legal transitions                    |
| final_score        | integer or null              | score_v1 terminal value                             | Non-negative; server calculated           |
| ranked_eligibility | enum / string                | eligible, unranked, invalidated                     | Official configuration required           |
| event_sequence     | integer                      | Durable room replay position                        | Monotonic and unique per room             |
| owner_id           | UUID                         | Supabase subject projected into app data            | Owner predicate on private queries        |
| key_version        | bounded string               | Selects encryption or HMAC key                      | Must exist in configured ring             |

### S4.6 Validation design

Validation occurs in layers. Pydantic checks transport shape, domain validation checks rule meaning, authorization checks ownership, and database constraints protect concurrent or bypassed application paths. The web client may help a player avoid mistakes, but server validation remains final.

# S5. Design Implementation

## S5.1 Initialization and configuration

The API loads typed settings, validates deployed security requirements, configures structured logging, tests Redis readiness, selects a Redis or in-memory broker according to environment, initializes bounded rate limiters, and starts deletion and room-finalization workers in staging or production. Alembic applies schema before API startup.

## S5.2 Data structures

| Structure         | Implementation use                                           | Reason                                                |
|-------------------|--------------------------------------------------------------|-------------------------------------------------------|
| Tuple             | Secrets, normalized codes, immutable attempt history in core | Prevents accidental in-place mutation                 |
| List              | Palette values, API JSON arrays, paginated response rows     | Ordered variable-length collection                    |
| Counter           | Unmatched colour frequency                                   | O(n) duplicate-safe feedback intersection             |
| Dataclass         | Core configuration, state, attempts, feedback                | Typed domain data without framework dependency        |
| Enum              | Mode, status, difficulty, Code Maker, eligibility            | Stable legal values and transition checks             |
| Dictionary / JSON | Persisted config and score breakdown                         | Versioned structured data across service boundaries   |
| Relational table  | Profiles, games, attempts, rooms, ranks, audits              | Constraints, transactions, indexes, durable ownership |

## S5.3 Core models in source

```

55 UNRANKED = "unranked"
56 INVALIDATED = "invalidated"
57
58
59 @dataclass(frozen=True, slots=True)
60 class GameConfig:
61     colours: tuple[str, ...]
62     code_length: int
63     max_attempts: int
64     duplicates_allowed: bool
65     code_maker: CodeMakerType = CodeMakerType.COMPUTER
66     visibility: GameVisibility = GameVisibility.PRIVATE
67     ranked: bool = False
68     time_bonus_cap: int = 300
69
70 def __post_init__(self) -> None:
71     normalized = tuple(str(colour).strip().upper() for colour in self.colours)
72     object.__setattr__(self, "colours", normalized)
73     if not 5 <= len(normalized) <= 10:
74         raise DomainError("INVALID_COLOUR_COUNT", "Choose between 5 and 10 colours.")
75     if len(set(normalized)) != len(normalized):
76         raise DomainError("DUPLICATE_COLOUR_OPTION", "Available colours must be unique.")
77     if any(colour not in COLOUR_IDS for colour in normalized):
78         raise DomainError("UNKNOWN_COLOUR", "The palette contains an unknown colour.")
79     if not 3 <= self.code_length <= 6:
80         raise DomainError("INVALID_CODE_LENGTH", "Code length must be between 3 and 6.")
81     if not 1 <= self.max_attempts <= 20:
82         raise DomainError("INVALID_MAX_ATTEMPTS", "Maximum attempts must be between 1 and 20.")
83     if not self.duplicates_allowed and len(normalized) < self.code_length:
84         raise DomainError(
85             "INSUFFICIENT_UNIQUE_COLOURS",
86             "Enable at least as many colours as there are code positions.",
87         )
88     if not 0 <= self.time_bonus_cap <= 3600:
89         raise DomainError("INVALID_TIME_BONUS_CAP", "Time bonus cap is out of range.")
90
91 def to_dict(self) -> dict[str, Any]:
92     return {
93         "colours": list(self.colours),
94         "codeLength": self.code_length,
95         "maxAttempts": self.max_attempts,
96         "duplicatesAllowed": self.duplicates_allowed,
97         "codeMaker": self.code_maker.value,
98         "visibility": self.visibility.value,
99         "ranked": self.ranked,
100         "timeBonusCap": self.time_bonus_cap,
101     }
102
103 @classmethod
104 def from_dict(cls, value: dict[str, Any]) -> GameConfig:
105     return cls(
106         colours=tuple(value["colours"]),
107         code_length=int(value["codeLength"] if "codeLength" in value else value["code_length"]),
108         max_attempts=int(
109             value["maxAttempts"] if "maxAttempts" in value else value["max_attempts"]
110         ),
111         duplicates_allowed=bool(
112             value.get("duplicatesAllowed", value.get("duplicates_allowed"))
113         ),
114         code_maker=CodeMakerType(value.get("codeMaker", value.get("code_maker", "computer"))),
115         visibility=GameVisibility(value.get("visibility", "private")),
116         ranked=bool(value.get("ranked", False)),
117         time_bonus_cap=int(value.get("timeBonusCap", value.get("time_bonus_cap", 300))),
118     )
119
120 @dataclass(frozen=True, slots=True)
121 class Feedback:
122     black: int

```

Supplement Code 1. Actual repository source, rendered in monochrome.

## S5.4 Classes and modules

| Class/module                                       | Responsibility                                             |
|----------------------------------------------------|------------------------------------------------------------|
| GameConfig, GameState, AttemptRecord, Feedback     | Framework-independent domain state and invariants          |
| Settings                                           | Typed environment contract and production validation       |
| Profile, GameSession, GameAttempt                  | Identity projection and authoritative gameplay persistence |
| FriendChallenge, MultiplayerRoom, MultiplayerEvent | Capability-based social play and durable replay            |
| SupabaseJWTVerifier, AuthPrincipal                 | Provider token verification and application principal      |
| SecretCipher                                       | Versioned AES-256-GCM encryption and decryption            |
| RedisBroker, InMemoryBroker, UnavailableBroker     | Environment-specific safe-event distribution               |
| GameBoard, SessionProvider                         | Typed interaction and browser identity lifecycle           |

## S5.5 Function catalogue

| Function               | Input                           | Output/effect                                  |
|------------------------|---------------------------------|------------------------------------------------|
| calculate_feedback     | secret, guess                   | Feedback with exact and misplaced counts       |
| generate_secret        | validated config, optional RNG  | Tuple satisfying duplicate rule                |
| submit_guess           | state, secret, guess, time, key | New immutable GameState                        |
| create_game service    | principal, request, cipher      | Encrypted persisted GameSession                |
| submit_attempt service | owned game, request             | Atomic attempt, terminal effects, response     |
| derive daily secret    | date, config, versioned HMAC    | Deterministic code without persisted plaintext |
| require_admin          | principal, database session     | Authorized principal or safe denial            |
| room event handler     | ticket, command, sequence       | Committed room state and safe event            |
| export account         | registered principal            | CSV-safe ZIP archive                           |

## S5.6 Libraries and dependencies

| Library/tool                                   | Purpose                                                              |
|------------------------------------------------|----------------------------------------------------------------------|
| FastAPI + Pydantic                             | HTTP routing, dependency injection, validation, OpenAPI              |
| SQLAlchemy + Alembic + asyncpg                 | Async persistence, transactions, schema migration                    |
| cryptography + PyJWT                           | AES-GCM and asymmetric JWT verification                              |
| Redis client + Prometheus + structlog          | Coordination, metrics, structured operational logs                   |
| Next.js + React + next-intl + Supabase client  | PWA routes, interaction, localization, identity                      |
| pytest + Hypothesis + Vitest + Playwright + k6 | Unit, property, component, browser, accessibility, and load evidence |

## S5.7 Test implementation example

```

1 from __future__ import annotations
2
3 import itertools
4
5 import pytest
6 from hypothesis import given
7 from hypothesis import strategies as st
8 from mastermind_core import DomainError, calculate_feedback
9
10
11 @pytest.mark.parametrize(
12     ("secret", "guess", "expected"),
13     [
14         ("RBGY", "RBGY", (4, 0)),
15         ("RBGY", "WKOP", (0, 0)),
16         ("RBGY", "RGBW", (1, 2)),
17         ("RBGY", "RRRR", (1, 0)),
18         ("RRBG", "RRRY", (2, 0)),
19         ("RBRG", "RRBY", (1, 2)),
20     ],
21 )
22 def test_mandatory_feedback_examples(secret: str, guess: str, expected: tuple[int, int]) -> None:
23     feedback = calculate_feedback(secret, guess)
24     assert (feedback.black, feedback.white) == expected
25
26
27 def test_exhaustive_duplicate_combinations() -> None:
28     colours = "RBG"
29     for secret in itertools.product(colours, repeat=4):
30         for guess in itertools.product(colours, repeat=4):
31             feedback = calculate_feedback(secret, guess)
32             assert 0 <= feedback.black <= 4
33             assert 0 <= feedback.white <= 4
34             assert feedback.black + feedback.white <= 4
35             assert feedback == calculate_feedback(secret, guess)
36
37
38 @given(
39     st.lists(st.sampled_from(list("RBGYWKOPCM")), min_size=1, max_size=10),
40     st.lists(st.sampled_from(list("RBGYWKOPCM")), min_size=1, max_size=10),
41 )
42 def test_feedback_invariants(secret: list[str], guess: list[str]) -> None:
43     if len(secret) != len(guess):
44         with pytest.raises(DomainError, match="lengths must match"):
45             calculate_feedback(secret, guess)
46     return
47     feedback = calculate_feedback(secret, guess)
48     assert 0 <= feedback.black <= len(secret)
49     assert 0 <= feedback.white <= len(secret)
50     assert feedback.black + feedback.white <= len(secret)
51
52
53 @given(st.lists(st.sampled_from(list("RBGYWKOPCM")), min_size=1, max_size=10))
54 def test_exact_equality_is_all_black(code: list[str]) -> None:
55     assert calculate_feedback(code, code).black == len(code)
56     assert calculate_feedback(code, code).white == 0

```

*Supplement Code 2. Actual repository source, rendered in monochrome.*

# S6. Testing and Evaluation Evidence

## S6.1 Testing plan

I divided testing into three phases. Phase 1 checks initialization, configuration, dependencies, and input boundaries. Phase 2 checks isolated algorithms and integrated API behavior. Phase 3 checks outputs, browser flows, accessibility, load, recovery, and user acceptance.

| Phase | Scope                                                                    | Method                                             | Status in this report                               |
|-------|--------------------------------------------------------------------------|----------------------------------------------------|-----------------------------------------------------|
| 1     | Imports, settings, keys, validation, CLI input                           | Static checks and targeted tests                   | Executed locally                                    |
| 2     | Core rules, API transactions, identity, rooms, retention                 | pytest, property cases, integration-style fixtures | Executed; external service cases skipped            |
| 3     | Web outputs, browser journeys, accessibility, containers, load, recovery | Vitest, Playwright, k6, drills, UAT                | Vitest executed; remaining release evidence pending |

## S6.2 Observed command evidence

```
$ uv run pytest
collected 141 items
133 passed, 8 skipped in 20.29s

$ uv run pytest tests/core --cov=mastermind_core --cov-branch
42 passed | 279 statements | 62 branches | 100.00% coverage

$ pnpm test
47 tests passed | web line coverage 88.4%

$ pnpm typecheck && uv run ruff check .
TypeScript passed | All Ruff checks passed
Warning: local Node 20.20.0; repository pins Node 24.18.0.
```

Supplement Figure 6. Command summaries recorded on 23 August 2026. The Node warning limits release equivalence.

## S6.3 Phase 1 cases: initialization and boundaries

| ID | Input or precondition                                          | Expected                           | Actual |
|----|----------------------------------------------------------------|------------------------------------|--------|
| T1 | Valid production-style AES key ring                            | Active version loads               | Passed |
| T2 | Unknown or malformed key version                               | Startup or decrypt fails safely    | Passed |
| T3 | JWT with wrong signature, issuer, audience, expiry, or subject | 401 safe error                     | Passed |
| T4 | Production URL points to local or mutable service              | Configuration rejected             | Passed |
| T5 | Guess is empty, malformed, wrong length, or unknown colour     | Stable domain error; no attempt    | Passed |
| T6 | Custom config breaks palette, duplicate, or bonus bounds       | Configuration rejected             | Passed |
| T7 | CLI receives invalid menu key, EOF, or interrupt               | No crash; safe exit or retry       | Passed |
| T8 | CSV has malformed row or formula-like field                    | Skip/warn and neutralize injection | Passed |

## S6.4 Phase 2 cases: algorithms and integrated behavior

| ID  | Case                                       | Expected                                     | Actual                             |
|-----|--------------------------------------------|----------------------------------------------|------------------------------------|
| T9  | Secret and guess match                     | All exact, zero misplaced                    | Passed                             |
| T10 | Duplicate-heavy exhaustive combinations    | No double counting; invariants hold          | Passed                             |
| T11 | Win on final attempt                       | Won state and valid score                    | Passed                             |
| T12 | Invalid guess against active state         | Original immutable state unchanged           | Passed                             |
| T13 | Retry same attempt key and payload         | Stored original outcome returned once        | Passed                             |
| T14 | Reuse idempotency key with another payload | Conflict; no second attempt                  | Passed                             |
| T15 | Read another player game by changed ID     | Owner-scoped denial                          | Passed                             |
| T16 | Start same daily twice                     | One official run; replay unranked            | Passed                             |
| T17 | Expired or revoked challenge token         | Generic not-found response                   | Passed                             |
| T18 | Reuse WebSocket ticket                     | Second use rejected                          | Passed                             |
| T19 | Both room members become ready             | Room starts once for both                    | Passed                             |
| T20 | Deletion provider fails then recovers      | Hidden data and durable retry complete later | Passed                             |
| T21 | Admin lacks grant, AAL2, or recent auth    | Administrative action denied                 | Passed                             |
| T22 | Retention worker encounters locked batch   | Skip locked rows and resume                  | Skipped without PostgreSQL service |

## S6.5 Phase 3 cases: outputs and acceptance

| ID  | Case                                       | Expected                                                   | Evidence status                        |
|-----|--------------------------------------------|------------------------------------------------------------|----------------------------------------|
| T23 | Game board component flow                  | Valid row submission, history, result states               | Vitest passed                          |
| T24 | Session and service-worker behavior        | Guest bootstrap and safe update handling                   | Vitest passed with known coverage gaps |
| T25 | English and Traditional Chinese catalogues | Required messages remain aligned                           | Catalogue tests passed                 |
| T26 | Keyboard and screen-reader gameplay        | All actions named, focus visible, feedback announced       | Playwright spec exists; not run here   |
| T27 | Mobile and reduced-motion behavior         | Usable narrow board and restrained transitions             | Playwright spec exists; not run here   |
| T28 | Two-client reconnect and replay            | No private guess leak; sequence catches up                 | Release smoke pending                  |
| T29 | Container build and migration from empty   | Non-root services start on current schema                  | CI workflow defined; not run here      |
| T30 | Load profile                               | Measured p95/p99 and saturation recorded                   | k6 scripts exist; measurements pending |
| T31 | Backup restore and key rotation            | Recovered data and old ciphertext remain usable            | Quarterly drills pending               |
| T32 | Student/user acceptance review             | New user completes solo, daily, challenge, account actions | Formal UAT pending                     |

## S6.6 Testing evaluation

The executed evidence supports domain correctness and a broad set of service behaviors. The eight skipped Python cases depend on real PostgreSQL or Redis conditions. Browser, container, load, recovery, and formal acceptance checks remain release gates. I record that distinction so the report does not turn a defined test into a claimed pass.

# S7. Debugging and Implemented Improvements

## S7.1 Debugging method

I used failing checks and repository diffs to record the symptom, cause, correction, and retest path. The six entries below come from committed fixes rather than reconstructed hypothetical bugs.

| Commit  | Observed problem                                           | Cause                                                                                                   | Correction and retest                                                                            |
|---------|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| 3efefdb | E2E browser hydration became unstable                      | Locale proxy intercepted Next internal HTTP/HMR routes and CI started the web server outside Playwright | Exclude <code>/_next</code> , let Playwright own web lifecycle, document behavior; rerun E2E job |
| c89a8db | Browser install command resolved from wrong workspace      | Root pnpm exec did not use the web package binary context                                               | Invoke pnpm --filter @mastermind/web exec playwright install                                     |
| f301a4e | Generated OpenAPI contract caused formatting drift         | Generator output was not normalized before git diff                                                     | Run Prettier in the generate script before drift check                                           |
| 79e0a81 | Security scan found an older PostCSS transitively selected | Next dependency graph retained 8.4.31                                                                   | Pin pnpm override to 8.5.19 and refresh lockfile                                                 |
| 1d52e1e | First-run CI and security checks failed                    | Early cache assumptions, action behavior, and encoded auth-link handling                                | Remove unsupported cache setup, use pinned Trivy container, normalize encoded links              |
| 6e18019 | Environment variants could evade ignore rules              | Only <code>.env</code> and <code>.env.local</code> were ignored                                         | Ignore <code>.env*</code> while explicitly retaining <code>.env.example</code>                   |

## S7.2 Implemented improvements

| Improvement                              | Result                                                                                        |
|------------------------------------------|-----------------------------------------------------------------------------------------------|
| Canonical engine extraction              | API and CLI share validation, feedback, transitions, daily derivation, and score logic.       |
| Anonymous authenticated onboarding       | A guest gains a real subject without account registration.                                    |
| Versioned encryption and scoring         | Old sessions remain decryptable and reproducible across key or algorithm changes.             |
| Durable real-time replay                 | A reconnecting room member resumes from PostgreSQL sequence events.                           |
| Accessible localized PWA                 | Keyboard, symbols, live announcements, reduced motion, mobile routes, and two locales.        |
| Retention and account-deletion lifecycle | Data hides first, external deletion retries, and scheduled cleanup remains bounded.           |
| Secure delivery workflows                | Pinned runtimes, migrations, security scans, non-root containers, and protected environments. |

## S7.3 Further improvement list

- Repeat the full browser and build matrix in Node 24.18.0 and keep artifacts with the release.
- Run real PostgreSQL and Redis concurrency cases, then record lock wait and replay evidence.
- Complete manual accessibility, formal user acceptance, restore, and key-rotation exercises.
- Refactor the densest service and room paths into smaller application services without changing transactions.

# S8. User Manual

## S8.1 System requirements

For the hosted product, a current browser with JavaScript and network access is sufficient. Local development requires Node 24.18.0, pnpm 10.28.2, Python 3.13.14 with uv, Docker Compose, and a Supabase project or local Auth stack. The standalone CLI requires Python and writes data/high\_scores.csv.

## S8.2 Starting the system

```
cp .env.example .env pnpm install --frozen-lockfile uv sync --frozen --all-extras pnpm dev:all # CLI only
uv run python -m mastermind_cli
```

## S8.3 Playing a solo game

- Open the locale home page and continue as a guest or sign in.
- Choose Play, select Easy, Normal, Hard, Expert, Practice, Custom, or pass-and-play, then start.
- Fill the current row with palette controls or keyboard input. Submit only when the row is complete.
- Read exact and misplaced feedback through symbols and labels. Use the history to eliminate possibilities.
- At the result screen, review score, secret disclosure, achievements, rank eligibility, and replay actions.

## S8.4 Daily, challenge, and duel workflows

| Mode             | Steps                                                                                                                        |
|------------------|------------------------------------------------------------------------------------------------------------------------------|
| Daily            | Open Daily, start the UTC puzzle, refresh safely if needed, complete one official run, view daily rank.                      |
| Friend challenge | Create rules and optional title, copy the one-time link, share privately, manage or revoke from My Challenges.               |
| Private duel     | Create a room, share the room code, both members join and mark ready, play independently, reconnect through sequence replay. |

## S8.5 Profiles, accessibility, and privacy controls

Profile pages show server-derived statistics, recent games, achievements, and public-name settings. Settings provide locale, appearance, symbols, and motion preferences. Account controls allow export and registered-account deletion. A player can remove a display name or opt out of public leaderboards without deleting gameplay history.

## S8.6 Troubleshooting

| Symptom                     | Action                                                                                            |
|-----------------------------|---------------------------------------------------------------------------------------------------|
| AUTH_NOT_CONFIGURED         | Set matching public Supabase values in web and issuer/audience values in API.                     |
| Readiness degraded          | Check database and Redis separately. Ranked and real-time writes may fail closed.                 |
| Game did not accept a guess | Check row length, palette values, connection, and whether the session is terminal or expired.     |
| Room reconnects repeatedly  | Return to the room page for a new one-use ticket; check Redis readiness and origin configuration. |
| Secret decryption error     | Operator restores the recorded key version. Do not create a replacement key.                      |
| Generated client drift      | Start API, run pnpm generate:api, inspect and commit intentional schema changes.                  |
| CLI score file warning      | Check data directory permissions and CSV header; the game should continue safely.                 |

## S8.7 Frequently asked questions

| Question                               | Answer                                                                                                              |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Do I need an account?                  | No. The product creates an authenticated guest identity. A linked account adds cross-device and lifecycle controls. |
| Can the browser reveal an active code? | No supported response, event, log, analytics event, or export contains the active plaintext.                        |
| Why is a custom game unranked?         | Official ranking requires an exact preset so players compete under the same rules.                                  |
| What happens offline?                  | The shell may load and preserve the current row, but authoritative gameplay waits for the API.                      |
| Can I delete my data?                  | Registered users can export and delete after exact confirmation and recent authentication.                          |

# S9. Program Evaluation and Student Reflection

## S9.1 Evaluation against objectives

| Objective                     | Assessment                          | Evidence or remaining gate                                      |
|-------------------------------|-------------------------------------|-----------------------------------------------------------------|
| O1 feedback correctness       | Met and verified                    | Exhaustive duplicate combinations; 100% core coverage           |
| O2 guest onboarding           | Implemented                         | Session and auth paths; release browser run remains             |
| O3 game modes                 | Implemented                         | Routes, models, tests, and web pages                            |
| O4 competitive fairness       | Met in design and local tests       | Server time, score, locks, eligibility, unique rows             |
| O5 restoration                | Implemented                         | Persistent attempts and durable room events; live smoke pending |
| O6 profiles and competition   | Implemented and tested              | Stats, awards, privacy, moderation tests                        |
| O7 export and deletion        | Implemented and tested              | ZIP exclusions, recent auth, retry worker                       |
| O8 accessibility/localization | Implemented; release review pending | Component/catalogue evidence; manual review pending             |
| O9 security                   | Strong implemented controls         | Crypto/auth/schema tests; managed deployment evidence pending   |
| O10 operations                | Documented and wired                | Runbooks and metrics; restore/load drills pending               |

## S9.2 Benefits and limitations

| Benefits                                                                     | Limitations                                                                       |
|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| One rule engine reduces disagreement between API and CLI.                    | The full product requires several managed dependencies and careful configuration. |
| Server authority and schema constraints protect official outcomes.           | Real concurrency guarantees need PostgreSQL and Redis integration evidence.       |
| Guest identity keeps onboarding short without trusting browser IDs.          | Anonymous users receive fewer durable/public account features by design.          |
| Accessibility, privacy, moderation, and recovery appear in the architecture. | Manual accessibility, legal review, load, restore, and key drills remain.         |
| Versioning supports future rule, score, daily, and key changes.              | The densest service and room modules demand careful maintenance.                  |

## S9.3 Student reflection

I began with a small feedback algorithm and learned that a trustworthy product depends on much more than the game loop. I had to decide where identity becomes trustworthy, where data becomes authoritative, and which failures should stop play. The project improved my understanding of immutable state, transactions, cryptography, async services, typed contracts, browser accessibility, and operational evidence.

Testing changed my approach to implementation. The best examples did more than confirm a result. They checked that invalid input left state unchanged, retries did not duplicate effects, private responses omitted secrets, and concurrent paths had a database guard. I also learned to separate a documented target from a measured result.

The next stage of my learning is production verification. I can read and test code locally, but managed TLS, real database locks, browser assistive technology, restore timing, and load saturation require controlled environments and recorded drills.

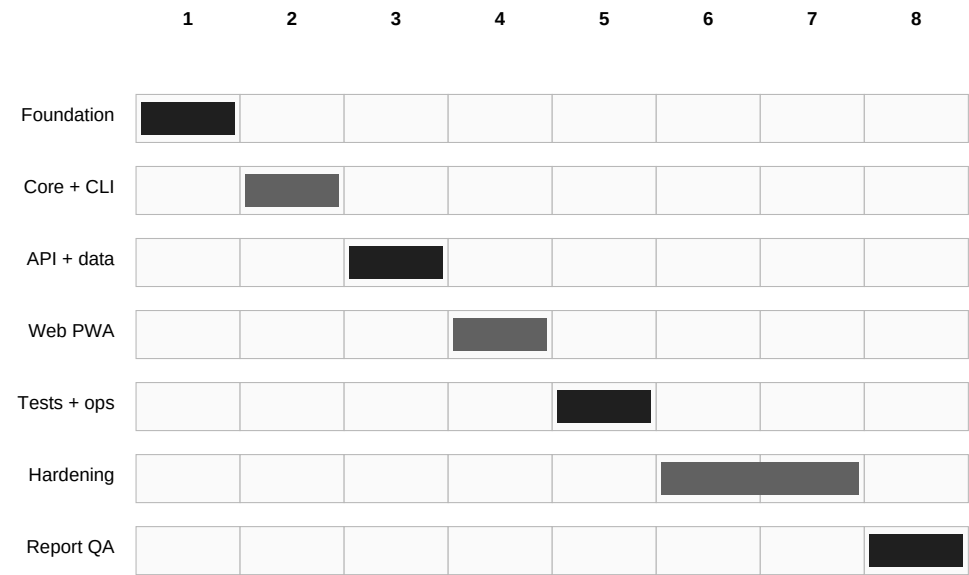
## S9.4 Future development

- Complete release-equivalent CI and staging evidence, then close every launch-checklist item.
- Add measured dashboards and tune alerts against real traffic without introducing high-cardinality or private labels.
- Extract smaller application services from the largest backend modules and retain transaction-focused tests.
- Expand locales and learning explanations only after the current accessibility contract passes manual review.

# S10. Project Management and Timeline

The repository records fourteen commits on 16 July 2026. Their order provides reliable phase evidence, but it does not prove how many calendar days each phase required. I therefore use ordered development stages rather than inventing durations. Report analysis and verification were refreshed on 23 August 2026.

Reconstructed project phase chart



Stages 1-7 are ordered from the commit sequence on 16 July 2026. Stage 8 records report verification on 23 August 2026.

Supplement Figure 7. A sequence-based Gantt view reconstructed from repository milestones.

## S10.1 Milestone record

| Stage | Repository evidence           | Deliverable                                                           |
|-------|-------------------------------|-----------------------------------------------------------------------|
| 1     | d0258a1, de7e14c              | Initialized history and production monorepo foundation                |
| 2     | 60070d3                       | Canonical engine and SBA CLI                                          |
| 3     | daa545d                       | Secure authoritative API, data, identity, games, rooms                |
| 4     | 572e0f4                       | Accessible localized web PWA                                          |
| 5     | a052879, 935f1fd, df38401     | Delivery workflows, coverage, operations, security, SBA documentation |
| 6     | 6e18019 through 3efefdb       | Environment, security, generation, Playwright, and hydration fixes    |
| 7     | 23 August verification record | Expanded report, test rerun, PDF visual QA                            |

## S10.2 Risk tracking

| Risk                                  | Response                                                 | Current state                               |
|---------------------------------------|----------------------------------------------------------|---------------------------------------------|
| Client manipulates score or secret    | Server authority and explicit response models            | Controlled in implementation                |
| Concurrent duplicate outcome          | Row lock, idempotency fingerprint, unique constraints    | Local tests; real DB cases pending          |
| Key loss or wrong version             | External versioned rings, backups, safe failure          | Control present; drill pending              |
| Accessibility regression              | Component/E2E specs and manual checklist                 | Automated subset passed; manual pending     |
| Deployment placeholder or secret leak | Launch validation, ignore rules, scanning, bundle checks | Workflows present; release evidence pending |

# S11. References, AI Use, and Evidence Integrity

## S11.1 Reference report use

The user supplied 'succesful report.pdf', a 134-page timetable-scheduling project report. I used its heading structure and evidence pattern to identify missing lifecycle sections. I did not copy its timetable content, code, test results, or conclusions into Cipherboard.

## S11.2 Cipherboard sources

| Source group                       | Evidence used                                                                 |
|------------------------------------|-------------------------------------------------------------------------------|
| README.md, PRODUCT.md              | Purpose, users, design principles, setup, validation, deployment              |
| docs/product and docs/architecture | Requirements, API, data, real-time, ranking, accessibility, localization      |
| docs/security and docs/privacy     | Threats, controls, data inventory, retention, user controls                   |
| docs/operations                    | Configuration, testing, deployment, backup, rollback, incidents, launch gates |
| apps, packages, migrations, tests  | Actual implementation, constraints, functions, routes, test cases             |
| Git history through 3efefdb        | Development sequence and six debugging records                                |

## S11.3 AI-tool disclosure

| Tool         | Date           | Use                                                                                                          | Human responsibility                                                                        |
|--------------|----------------|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| OpenAI Codex | 23 August 2026 | Compared report structures, inspected repository evidence, ran commands, drafted and rendered the supplement | The student must review claims, personalize identity details, and accept the submitted work |

I did not use AI output as proof that a feature works. Test commands, source code, migrations, Git diffs, and rendered PDF inspection provide the evidence cited in this supplement. Unrun checks remain marked pending.

## S11.4 Evidence-integrity statement

- Requirements and performance targets remain distinct from measurements.
- Skipped tests and unexecuted release checks do not count as passes.
- The project timeline uses commit order and does not invent effort duration.
- The reference report influenced coverage only; Cipherboard claims come from its own repository.

# End of Report

## Cipherboard Complete System Report with Development Evidence Supplement

The expanded report now includes the full product and architecture assessment in Part I and the student development lifecycle, detailed verification, debugging record, user manual, evaluation, timeline, and AI-use disclosure in Part II.

**Final evidence status**

Local static and unit checks passed. Release-equivalent browser, managed-service, load, recovery, legal, and manual accessibility gates remain identified as pending.